

LANGUAGE PL/SQL

Langage PL/SQL



PLAN

- ☐ Introduction
- ☐ Avantages de PL/SQL
- ☐ Les Principales caractéristique du Pl / SQL
- ☐ Les Variables et les constantes
- ☐ Les instructions de base
- ☐ Interaction avec la base de données à partir de PL/SQL
- ☐ Les curseurs
- ☐ Gestion de exceptions
- ☐ Les procédures, fonctions
- ☐ Les triggers

Introduction



- ❑ PL/SQL (Procedural Language / SQL), l'extension procédural proposée par Oracle pour SQL,
- ❑ Il permet de combiner des requêtes SQL (SELECT, INSERT, UPDATE et DELETE) et des instructions procédurales (boucles, conditions...),
- ❑ Créer des traitements complexes destinés à être stockés sur le serveur de base de données
- ❑ Comme on le sait, les structures de contrôle habituelles d'un langage (IF, WHILE..) ne font pas partie intégrante de la norme SQL. Oracle les prend en compte dans PL/SQL.

Introduction

Id_Produit	Libellé	Marque	Prix	ID_Fournisseur
P1	HP600	HP	1000	1
P2	Epson30	Epson	2300	1
P3	IBM 7380	IBM	4600	2
P4	Lexmark310	Lexmark	2000	3

❑ Update produit set prix=2500 where id_fournisseur=1

CODE PL/SQL

Procedure :

IF ... THEN

ELSEIF

ELSE

END IF

❑ Pour le fournisseur N°1

Si Moyenne(prix)>2000

Augmenter le prix de 5%

Sinon

Augmenter le prix de 3%

Avantage du langage PL/SQL

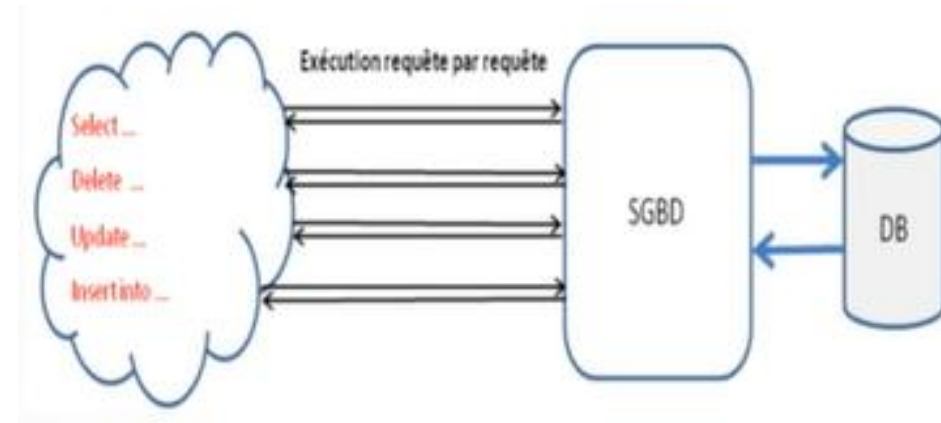


Ce langage propose des-performances pour le traitement des transactions et offre les avantages suivants :

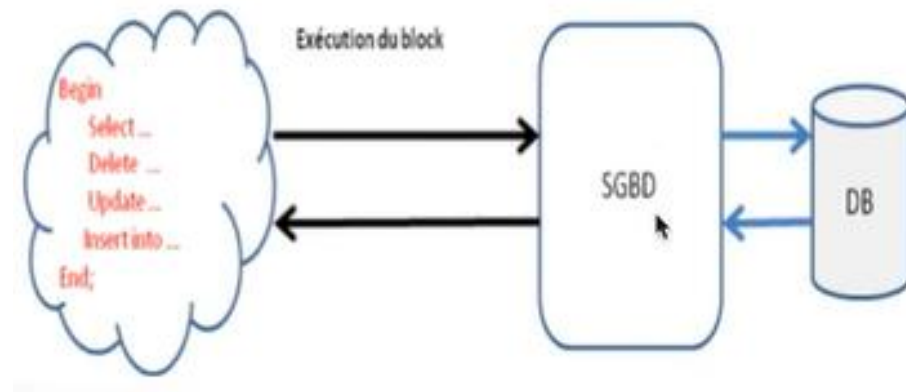
- ❑ Intégration complète du SQL
- ❑ Prise en charge de la programmation orientée objet (surcharge),
- ❑ Parfaite intégration avec Oracle et Java, en fait **PL/SQL** est un langage propriétaire de Oracle,
 - On peut lancer des sous-programme PL/SQL à partir de **Java** et de même, on peut appeler des procédures Java à partir d'un block PL/SQL

Avantage du langage PL/SQL

Dans un environnement client-serveur, chaque instruction SQL donne lieu à l'envoi d'un message du client vers le serveur suivi de la réponse du serveur vers le client



Un bloc PL/SQL donne lieu à un seul échange sur le réseau entre le client et le serveur. Les résultats intermédiaires sont coté serveur et seul le résultat final est retourné au client.



Langage PL/SQL



Un programme ou une procédure PL/SQL est un ensemble d'un ou plusieurs blocs. Chaque bloc comporte trois sections :

- Section déclaration
- Section Corps du bloc
- Section traitement des erreurs (exceptions)

Structure de bloc PL/SQL

[DECLARE] (facultatif)

Déclarer les objets PL/SQL à utiliser dans ce bloc

BEGIN (obligatoire)

Définir les instructions exécutables

[EXCEPTION] (facultatif)

Définir les actions à entreprendre en cas d'erreur ou d'exception

END; (obligatoire)

Sub Block

DECLARE

BEGIN

EXCEPTION

END;

Structure de bloc PL/SQL

Les types des blocs

Anonymous

```
[DECLARE]

BEGIN
    --statements

[EXCEPTION]

END;
```

Procedure

```
PROCEDURE name
IS

BEGIN
    --statements

[EXCEPTION]

END;
```

Function

```
FUNCTION name
RETURN datatype
IS

BEGIN
    --statements

[EXCEPTION]

END;
```

Outils de programmation PL/SQL

```
C:\> sqlplus
```

```
DECLARE
```

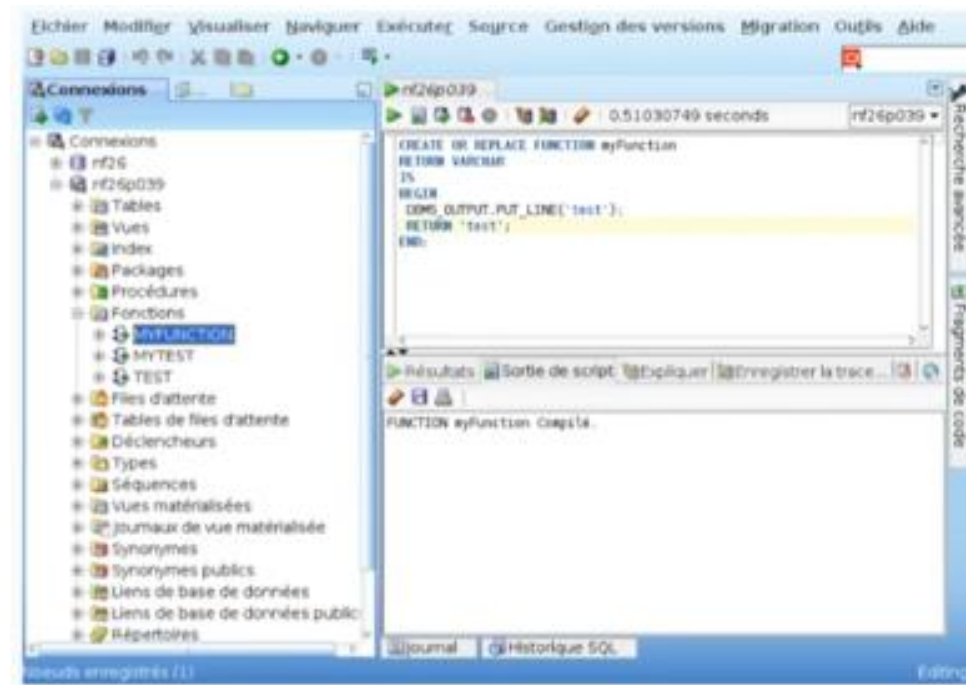
```
    v_telephone NUMBER(10);  
    v_disponible BOOLEAN;  
    v_DateDeNaissance Date;  
    c_TVA CONSTANT NUMBER :=0.02;  
    v_TTC NUMBER ;  
    v_HT NUMBER ;
```

```
BEGIN
```

```
    v_disponible :=TRUE ;  
    v_telephone :=0666666666;  
    v_TTC := v_HT + c_TVA ;
```

```
END;  
/
```

Outil: SQLPLUS



Outil: SQL Developer

Mon premier programme PL/SQL

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
  DBMS_OUTPUT.put_line('Bonjour tout le monde');
```

```
END;
```

```
/
```



PL/SQL

Déclaration des variables

Déclaration des variables

- Les types habituels correspondants au SQL2 ou Oracle : **integer, varchar, ...**
- Les types composites adaptés à la récupération des colonnes et des lignes des SQL ; **%TYPE, %ROWTYPE**

Règles :

- Adopter les conventions de dénomination des objets.
- Initialiser les constantes et les variables déclarées **NOT NULL**.
- Initialiser les identifiants en utilisant l'opérateur d'affectation (**:=**) ou le mot réservé **DEFAULT**
- Déclarer au plus un identifiant par ligne.

Déclaration des variables

Syntaxe:

identifier [CONSTANT] *datatype* [NOT NULL]
[:= | DEFAULT *exp*];

Exemple :

DECLARE

```
v_DateNaissance    DATE;
v_NDept            Number(2) NOT NULL :=10;
v_Ville            VARCHAR2(13) := 'Aix';
v_CodeP            CONSTANT NUMBER := 13100 ;
v_Valide           BOOLEAN NOT NULL := TRUE;
v_greet            VARCHAR2(20) DEFAULT 'have a Goode day';
```

Déclaration des variables

Les types d'oracle

TYPE	VALEURS
BINARY-INTEGER	entiers allant de -2^{31} à 2^{31}
POSITIVE / NATURAL	entiers positifs allant jusqu'à $2^{31} - 1$
BUMBER	Numérique entre -2^{418} à 2^{418}
INTEGER	Entier stocké en binaire entre -2^{126} à 2^{126}
CHAR (n)	Chaine fixe de 1 32767 caractères (différent pour une colonne de table)
VARCHAR2 (n)	Chaine variable de 1 à 32767 caractères
LONG	Idem VARCHAR2 (maximum 2 gigaoctet)
RAW	Permet de stocker des types de données binaire relativement faibles (≤ 32767 octet) idem VARCHAR2
LON RAW	Idem LONG mais avec du binaire
CLOB	Données d'objet de grande taille de type caractère. Objets de type long stockés en binaire (maximum 4 giga octets)

Déclaration des variables avec l'attribut %Type

Exemple :

```
...  
    v_Nom          client.nom%TYPE;  
    v_age          NUMBER(7,2);  
    v_min_age      v_age%TYPE := 10;  
...
```

RQ : NOT NULL ne s'applique pas aux variables déclarées avec %TYPE

Créer un enregistrement PL/SQL

Déclare des variables pour le stockage du nom, du poste et du salaire d'un nouvel employé

Exemple :

```
...  
TYPE emp_record_type IS RECORD  
  ( nom      VARCHAR2(25) ,  
    fonction  VARCHAR2(10) ,  
    salaire   NUMBER(8,2)  
  );  
emp_record   emp_record_type;
```

RQ : - NOT NULL peut être appliqué au champ
- champ peut être déclaré avec %TYPE

Entrée-sortie PL/SQL

Il est évidemment possible d'utiliser des fonctions prédéfinies d'entrée/sortie. Ces fonctions servent essentiellement durant la phase de test de procédure PL/SQL

Pour entrer une valeur : **ACCEPT** msg **PROMPT**

Pour afficher une valeur : **DBMS_OUTPUT.put_line**('Test : ' || message);;

```
ACCEPT msg PROMPT 'Entrer votre message'  
DECLARE  
  message varchar2(100):='&msg';  
BEGIN  
  DBMS_OUTPUT.put_line(TEST: '|| message'  
END;
```



Exemples : Variable

The screenshot displays the SQL Developer interface with the following components:

- Menu Bar:** Fichier, Modifier, Affichage, Naviguer, Exécuter, Source, Equipe, Outils, Fenêtre, Aide.
- Toolbars:** Standard icons for file operations and SQL execution.
- Left Panel:**
 - Connexions:** Oracle connexions, mydb, ORCL, Connexions au service de schéma.
 - Rapports:** Tous les rapports, Rapports Data Modeler, Rapports de dictionnaire de données, Rapports définis par l'utilisateur, Rapports de vue analytique, Rapports OLAP, Rapports TimesTen.
- Top Tab Bar:** Page de bienvenue, variables.sql, mydb.
- SQL Worksheet:**
 - Feuille de calcul SQL:** Historique, 0,114 secondes.
 - Feuille de calcul:** Query Builder.
 - Code:**

```
DECLARE
var_nom VARCHAR2(20);
var_date DATE;
Var_age NUMBER;
BEGIN
var_nom := 'HADID';
var_age := 23;
var_date := '01/04/2020';
DBMS_OUTPUT.PUT_LINE('Le nom : ' || var_nom);
DBMS_OUTPUT.put_LINE('La date : ' || var_date);
DBMS_OUTPUT.PUT_LINE('L'age : ' || var_age);
END;
```
- Bottom Panel:**
 - Sortie de script:** Tâche terminée en 0,114 secondes.
 - Output:**

```
Le nom :HADID
La date :01/04/20
L'age :23

Procédure PL/SQL terminée.
```

Exemples : Variable

The screenshot displays the Oracle SQL Developer environment. The top window, titled 'Feuille de calcul SQL', contains a PL/SQL script. The script declares three variables: `var_nom` of type `VARCHAR2(20)` with value `'TOTO'`, `var_date` of type `DATE` with value `'02/03/2019'`, and `Var_age` of type `NUMBER` with value `30`. It then uses `DBMS_OUTPUT.PUT_LINE` to print the values of these variables. The bottom window, titled 'Sortie de script', shows the output of the script execution, which matches the declared values. The status bar indicates the task was completed in 0.093 seconds.

```
set SERVEROUTPUT ON
DECLARE
  var_nom VARCHAR2(20):='TOTO';
  var_date DATE:='02/03/2019';
  Var_age NUMBER :=30;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Le nom : ' || var_nom);
  DBMS_OUTPUT.put_LINE('La date : ' || var_date);
  DBMS_OUTPUT.PUT_LINE('L'age : ' || var_age);
END;
```

Sortie de script x

Tâche terminée en 0,093 secondes

Le nom :TOTO
La date :02/03/19
L'age :30

Procédure PL/SQL terminée.

Exemples : Variable

The screenshot displays the Oracle SQL Developer environment. The top window, titled 'Feuille de calcul SQL', contains a PL/SQL script. The script declares three variables: `var_nom` of type `VARCHAR2(20)` with value `'TOTO'`, `var_date` of type `DATE` with value `'02/03/2019'`, and `Var_age` of type `NUMBER` with value `30`. It then uses `DBMS_OUTPUT.PUT_LINE` to print the values of these variables. The bottom window, titled 'Sortie de script', shows the output of the script execution, which matches the declared values. The task was completed in 0.093 seconds.

```
set SERVEROUTPUT ON
DECLARE
  var_nom VARCHAR2(20):='TOTO';
  var_date DATE:='02/03/2019';
  Var_age NUMBER :=30;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Le nom : ' || var_nom);
  DBMS_OUTPUT.put_LINE('La date : ' || var_date);
  DBMS_OUTPUT.PUT_LINE('L'age : ' || var_age);
END;
```

Sortie de script x

Tâche terminée en 0,093 secondes

Le nom :TOTO
La date :02/03/19
L'age :30

Procédure PL/SQL terminée.

Exemples : Variable %TYPE

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connexions' pane shows a tree view for the 'mydb' connection, including tables like 'EMPLOYEES' with columns 'ID', 'NAME', 'DEPARTMENT', and 'SALARY'. The main window is titled 'Feuille de calcul SQL' and shows a script named 'variable-ôPE.sql'. The script declares variables using the %TYPE attribute: 'var_nom employees.name%TYPE;', 'var_date DATE := '02/03/2025';', and 'Var_age NUMBER :=30;'. It then assigns 'TITI' to 'var_nom' and uses 'DBMS_OUTPUT.PUT_LINE' to display the values. The bottom pane, 'Sortie de script', shows the execution results: 'Le nom :TITI', 'La date :02/03/25', and 'L'age :30'. The status bar indicates 'Tâche terminée en 0,046 secondes'.

```
set SERVEROUTPUT ON
DECLARE
  var_nom employees.name%TYPE;
  var_date DATE := '02/03/2025';
  Var_age NUMBER :=30;
BEGIN

  var_nom:= 'TITI';

  DBMS_OUTPUT.PUT_LINE('Le nom : ' || var_nom);
  DBMS_OUTPUT.put_LINE('La date : ' || var_date);
  DBMS_OUTPUT.PUT_LINE('L'age : ' || var_age);
END;
```

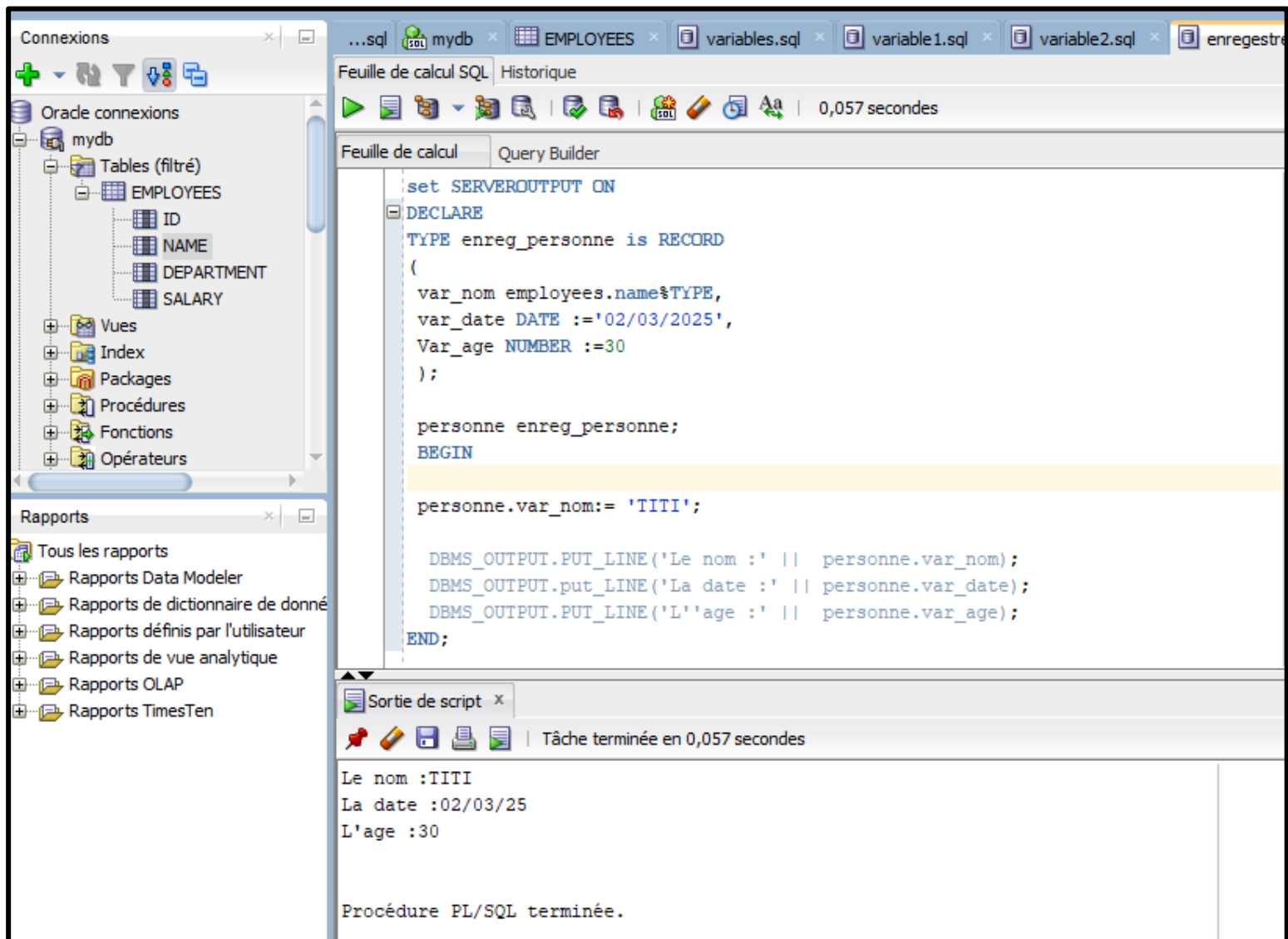
Sortie de script x

Tâche terminée en 0,046 secondes

Le nom :TITI
La date :02/03/25
L'age :30

Procédure PL/SQL terminée.

Exemples : Variable (enregistrement)



The screenshot displays the Oracle SQL Developer environment. On the left, the 'Connexions' pane shows a connection to 'mydb' with a tree view of database objects including 'EMPLOYEES' (with columns ID, NAME, DEPARTMENT, SALARY), 'Vues', 'Index', 'Packages', 'Procédures', 'Fonctions', and 'Opérateurs'. The 'Rapports' pane lists various report types. The main window shows a SQL script in the 'Feuille de calcul SQL' editor, with a toolbar and a status bar indicating '0,057 secondes'. The script is as follows:

```
set SERVEROUTPUT ON
DECLARE
TYPE enreg_personne is RECORD
(
  var_nom employees.name%TYPE,
  var_date DATE := '02/03/2025',
  Var_age NUMBER := 30
);

personne enreg_personne;
BEGIN

personne.var_nom:= 'TITI';

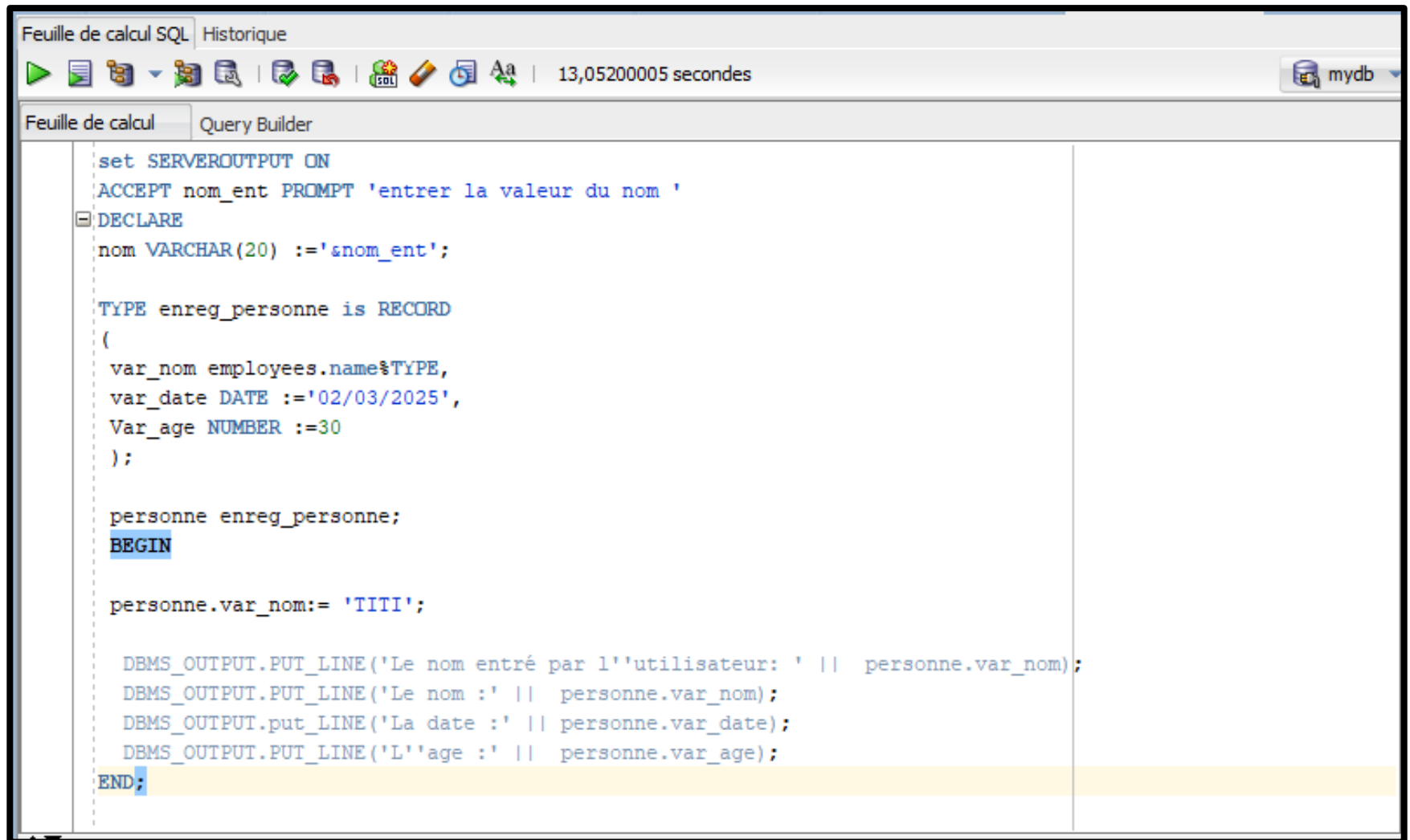
DBMS_OUTPUT.PUT_LINE('Le nom : ' || personne.var_nom);
DBMS_OUTPUT.put_LINE('La date : ' || personne.var_date);
DBMS_OUTPUT.PUT_LINE('L'age : ' || personne.var_age);
END;
```

Below the script editor, the 'Sortie de script' pane shows the execution results:

```
Le nom :TITI
La date :02/03/25
L'age :30

Procédure PL/SQL terminée.
```

Exemples : Variable (Entrer)



The screenshot shows a SQL IDE window with a toolbar and a code editor. The toolbar includes icons for running, saving, and other database operations, along with a timer showing 13,05200005 secondes. The code editor contains the following PL/SQL script:

```
set SERVEROUTPUT ON
ACCEPT nom_ent PROMPT 'entrer la valeur du nom '
DECLARE
nom VARCHAR(20) := '&nom_ent';

TYPE enreg_personne is RECORD
(
var_nom employees.name%TYPE,
var_date DATE := '02/03/2025',
Var_age NUMBER := 30
);

personne enreg_personne;
BEGIN

personne.var_nom:= 'TITI';

DBMS_OUTPUT.PUT_LINE('Le nom entré par l'utilisateur: ' || personne.var_nom);
DBMS_OUTPUT.PUT_LINE('Le nom : ' || personne.var_nom);
DBMS_OUTPUT.put_LINE('La date : ' || personne.var_date);
DBMS_OUTPUT.PUT_LINE('L'age : ' || personne.var_age);
END;
```



PL/SQL

Les instructions de base

Instructions de base

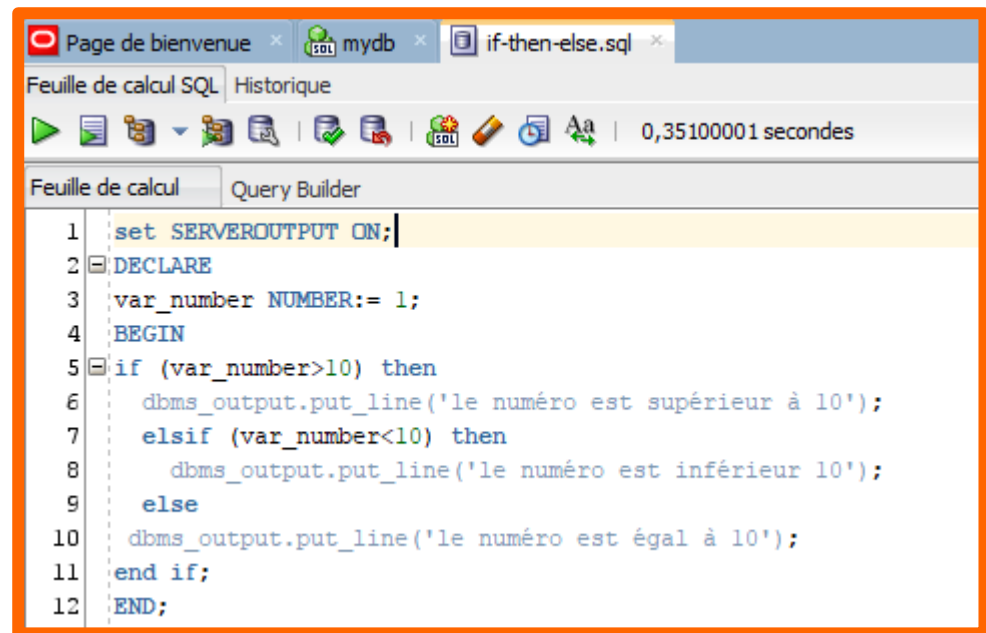
IF-THEN-ELSE

L'instruction IF-ELSE est une des premières structures conditionnelles que l'on apprend en programmation. Elle est composée d'une condition. Si cette condition est vraie, alors les instructions après le IF sont exécutées. Si la condition est fausse, ce sont les instructions sous le ELSE qui sont exécutées.

Exemple :

```
. . .
IF v_start > 100 THEN
    v_start := 2 * v_start;

ELSIF v_start >= 50 THEN
    v_start := 0.5 * v_start;
ELSE
    v_start := 0.1 * v_start;
END IF;
. . .
```

A screenshot of a SQL IDE window. The title bar shows 'Page de bienvenue', 'mydb', and 'if-then-else.sql'. The interface includes a 'Feuille de calcul SQL' tab, a 'Historique' panel, and a toolbar with icons for running queries, saving, and editing. The main editor area shows a SQL script with line numbers 1 through 12. The script sets SERVEROUTPUT ON, declares a variable, and uses an if-then-else structure to output messages based on the variable's value. The status bar at the bottom indicates a execution time of 0,35100001 secondes.

```
1 set SERVEROUTPUT ON;
2 DECLARE
3 var_number NUMBER:= 1;
4 BEGIN
5 if (var_number>10) then
6     dbms_output.put_line('le numéro est supérieur à 10');
7 elseif (var_number<10) then
8     dbms_output.put_line('le numéro est inférieur 10');
9 else
10    dbms_output.put_line('le numéro est égal à 10');
11 end if;
12 END;
```

Instructions de base

Instruction Case

Dans le langage SQL, la commande "CASE ...WHEN.." permet d'utiliser des conditions de type "si / sinon" similaire à un langage de programmation pour retourner un résultat disponible entre plusieurs possibilités

Exemple :

```
DECLARE
Mention char(1);
BEGIN
CASE
    WHEN Mention ='T' THEN
        DBMS_OUTPUT.PUT_LINE('Mention Très bien');
    WHEN Mention ='B' THEN
        DBMS_OUTPUT.PUT_LINE('Mention bien');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Mention Passable');
    END CASE;
END;
```

Instructions de base

Instruction Case

Dans le langage SQL, la commande "CASE ...WHEN.." permet d'utiliser des conditions de type "si / sinon" similaire à un langage de programmation pour retourner un résultat disponible entre plusieurs possibilités

Exemple :

A screenshot of a SQL Query Builder window. The window has two tabs: 'Feuille de calcul' and 'Query Builder'. The 'Query Builder' tab is active, showing a SQL script. The script is as follows:

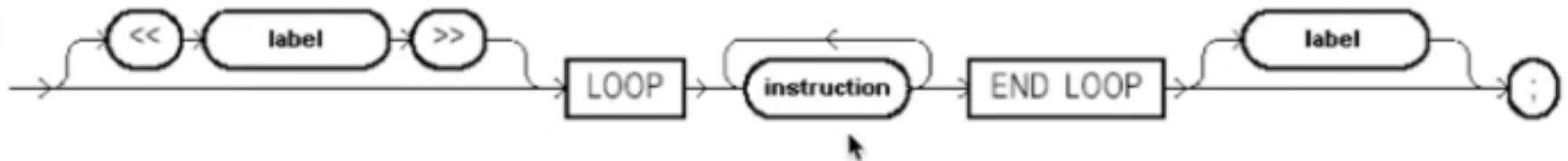
```
1 set SERVEROUTPUT ON;
2 DECLARE
3   var_note NUMBER:= 11;
4   BEGIN
5   CASE
6     when var_note<10 then
7       dbms_output.put_line('la mention: mal');
8
9     when var_note>=10 and var_note<12 then
10      dbms_output.put_line('la mention: passable');
11
12    when var_note>=12 and var_note<14 then
13      dbms_output.put_line('la mention: Assez bien');
14
15    when var_note>=14 and var_note<16 then
16      dbms_output.put_line('la mention: Bien');
17
18    else
19      dbms_output.put_line('la mantion : Très Bien');
20  end CASE;
21  END;
```

The script is numbered 1 through 21. The 'END;' statement at line 21 is highlighted in yellow. The window title bar shows 'Feuille de calcul' and 'Query Builder'.

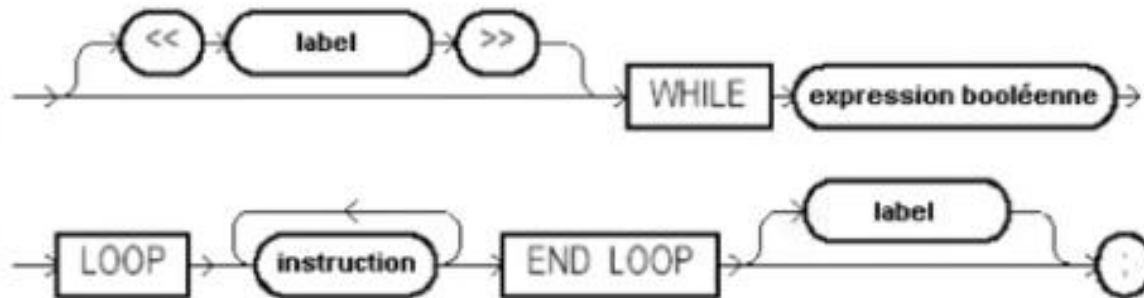
Instructions de base

Les boucles

Boucle LOOP



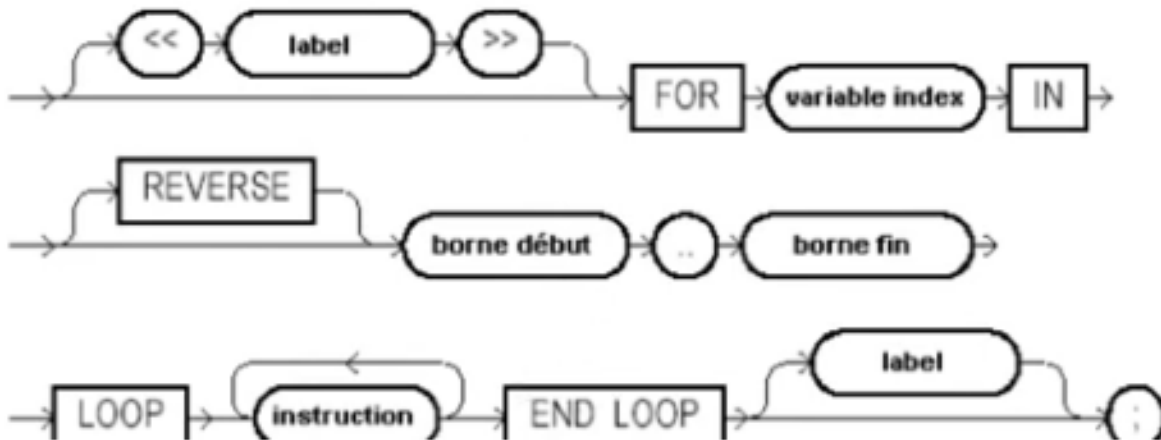
Boucle WHILE



Instructions de base

Les boucles

Boucle FOR



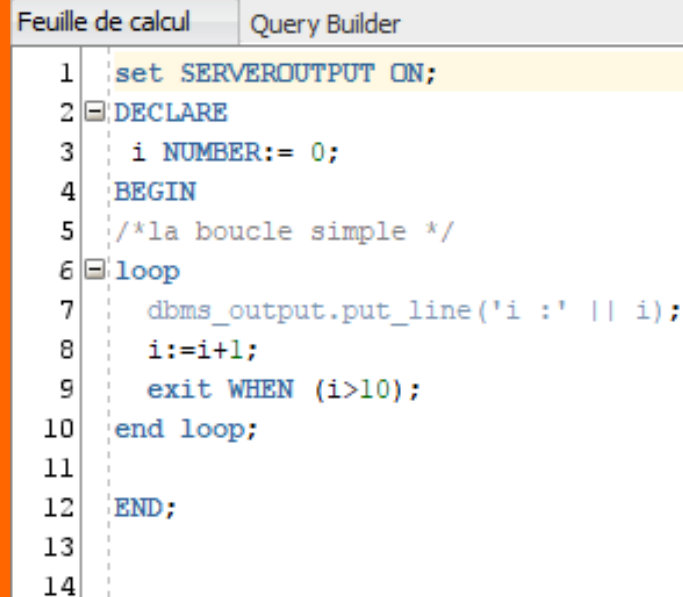
Instructions de base

Boucle simple

Exemple :

```
DECLARE
    Compte Number;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(Compte);

        compte := compte + 1;
        EXIT WHEN compte > 10;
    END LOOP;
END;
```



The screenshot shows a SQL query editor window with two tabs: 'Feuille de calcul' and 'Query Builder'. The 'Query Builder' tab is active, displaying a SQL script. The script is as follows:

```
1 set SERVEROUTPUT ON;
2 DECLARE
3     i NUMBER:= 0;
4 BEGIN
5     /*la boucle simple */
6 loop
7     dbms_output.put_line('i :' || i);
8     i:=i+1;
9     exit WHEN (i>10);
10 end loop;
11
12 END;
13
14
```

Instructions de base

Boucle FOR

Exemple :

```
DECLARE

BEGIN
  FOR i IN 1..10 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
```

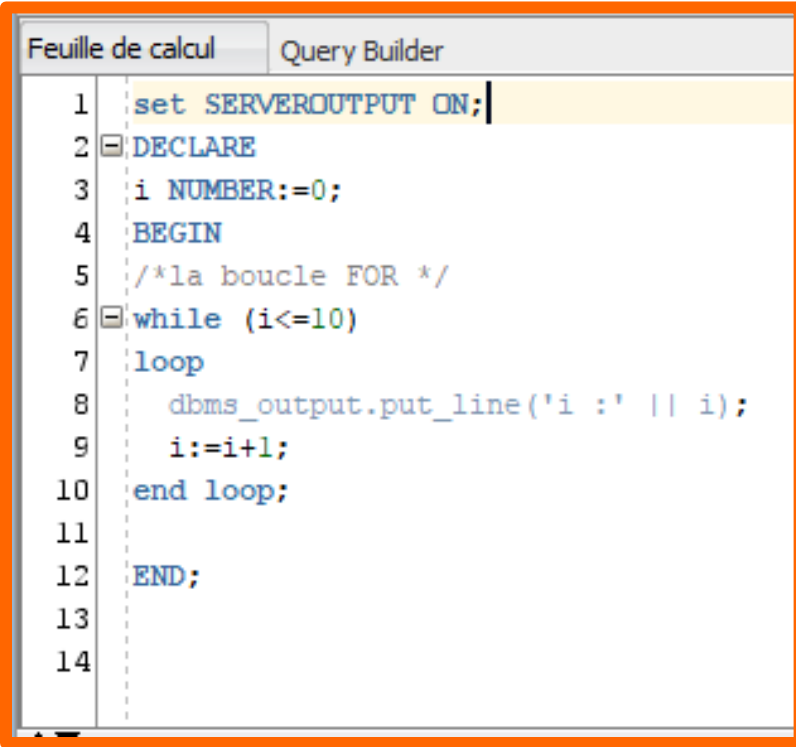
	Feuille de calcul	Query Builder
1		set SERVEROUTPUT ON;
2	☐	DECLARE
3		
4		BEGIN
5		/*la boucle FOR */
6	☐	for i in 1..10
7		loop
8		dbms_output.put_line('i :' i);
9		end loop;
10		
11		END;
12		

Instructions de base

Boucle WHILE

Exemple :

```
DECLARE
i      INTEGER;
BEGIN
  While i < 10 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
    i:=i+1;
  END LOOP;
END;
```



The screenshot shows a SQL query editor window with two tabs: "Feuille de calcul" and "Query Builder". The "Query Builder" tab is active, displaying a SQL script. The script is as follows:

```
1  set SERVEROUTPUT ON;
2  DECLARE
3  i NUMBER:=0;
4  BEGIN
5  /*la boucle FOR */
6  while (i<=10)
7  loop
8      dbms_output.put_line('i :' || i);
9      i:=i+1;
10 end loop;
11
12 END;
13
14
```



PL/SQL

Interaction avec la base de données
Les collections

Interaction avec la base de base

La clause INTO en PL/SQL

La clause **INTO** permet de passer des valeurs d'une table dans des variables

Exemple :

```
DECLARE
    v_id NUMBER(4);
    v_ville varchar(20);
BEGIN
    SELECT id_Emp, ville_Emp
    INTO v_id, v_ville
    FROM Employe
    WHERE nom_Emp= 'Ahmadi';
    ...
END;
```

Interaction avec la base de base

Mettre à jour des données

Augmenter le salaire de tous les employés du service **commercial** dans la table EMPLOYE

Exemple :

```
DECLARE
    v_salaire    employe.salaire%TYPE := 2000;
BEGIN
    UPDATE employe
    SET    salaire = salaire + v_salaire
    WHERE  service= 'commercial';
END;
```

RQ: En l'absence de modification de ligne, aucune erreur ne se produit

Interaction avec la base de base

Supprimer les données

Supprimer les lignes appartenant **service achat** à partir de la table EMPLOYE

Exemple :

```
DECLARE
    v_service    employe.service%TYPE := 'achat';
BEGIN
    DELETE FROM employees
    WHERE service = v_service;
END;
```

Instructions COMMIT et ROLLBACK



- ❑ Initialiser une transaction avec la première instruction **LMD** suivant **COMMIT** ou **ROLLBACK**
- ❑ Utiliser les instructions **SQL COMMIT** ou **ROLLBACK** pour mettre fin explicitement à une transaction

Déclarer des variables avec l'attribut %ROWTYPE

Déclarer une variable permettant de stocker les mêmes informations relatives à un employé telles qu'elles figurent dans la table EMPLOYEES.

Exemple :

```
DECLARE
    Emp_record  Employees%ROWTYPE;
    ...
BEGIN
    SELECT * INTO Emp_record
    FROM    Employees
    WHERE   ...
```

Déclarer des variables avec l'attribut %ROWTYPE

Exemple :

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connexions' pane shows the 'mydb' connection and the 'EMPLOYEES' table structure with columns ID, NAME, DEPARTMENT, and SALARY. The main window is titled 'Feuille de calcul SQL' and contains a script in the 'RowType1.sql' file. The script declares a variable `v_employee` of type `EMPLOYEES%ROWTYPE`, selects data for employee ID 2, and outputs the details using `dbms_output.put_line`. The 'Sortie de script' pane at the bottom shows the execution results, confirming the employee's details and stating 'Procédure PL/SQL terminée.'

```
DECLARE
v_employee EMPLOYEES%ROWTYPE;

BEGIN
select * into v_employee from employees where id=2;
dbms_output.put_line('l'identifiant de l'employee :' || v_employee.id);
dbms_output.put_line('le nom de l'employee :' || v_employee.name);
dbms_output.put_line('le departement de l'employee :' || v_employee.department);
dbms_output.put_line('le salaire l'employee :' || v_employee.salary);

END;
```

Sortie de script x

Tâche terminée en 0,152 secondes

l'identifiant de l'employee :2
le nom de l'employee :N2
le departement de l'employee :D2
le salaire l'employee :20

Procédure PL/SQL terminée.

Gestion des tableaux

Un tableau en PL/SQL est en fait une table qui ne comporte qu'une seule colonne. Cette colonne ne peut correspondre qu'à un type scalaire et nom à un type composé (table ou record)

La structure de données nécessaire pour gérer une table ORACLE nécessitera donc autant de types tables que la table possède de colonnes. On pourra éventuellement regrouper ces tables à l'intérieur d'un RECORD (Enregistrement)

0	1	2	3	4	5
Text0	Text1	Text2	Text3	Text4	Text5

```
TYPE tableau IS TABLE OF VARCHAR(20)  
INDEX BY BINARY_INTEGER;
```

Gestion des tableaux

```
DECLARE
    TYPE    nom_type_tableau    IS TABLE OF VARCHAR(20)
                                INDEX BY BINARY_INTEGER;
-- Déclaration de la variable tab de type nom_type_tableau

    tab    nom_type_tableau ;

BEGIN
    tab(1) := 'ligne 1' ;
    tab(2) := 'ligne 2' ;

    -- Affichage du premier élément de la variable tab,
    -- c'est à dire "ligne 1"
    IF tab.EXISTS(1) THEN
        DBMS_OUTPUT.PUT_LINE(tab(1)) ;
    END IF;

END ;
```

Gestion des tableaux

Les fonctions d'un tableau

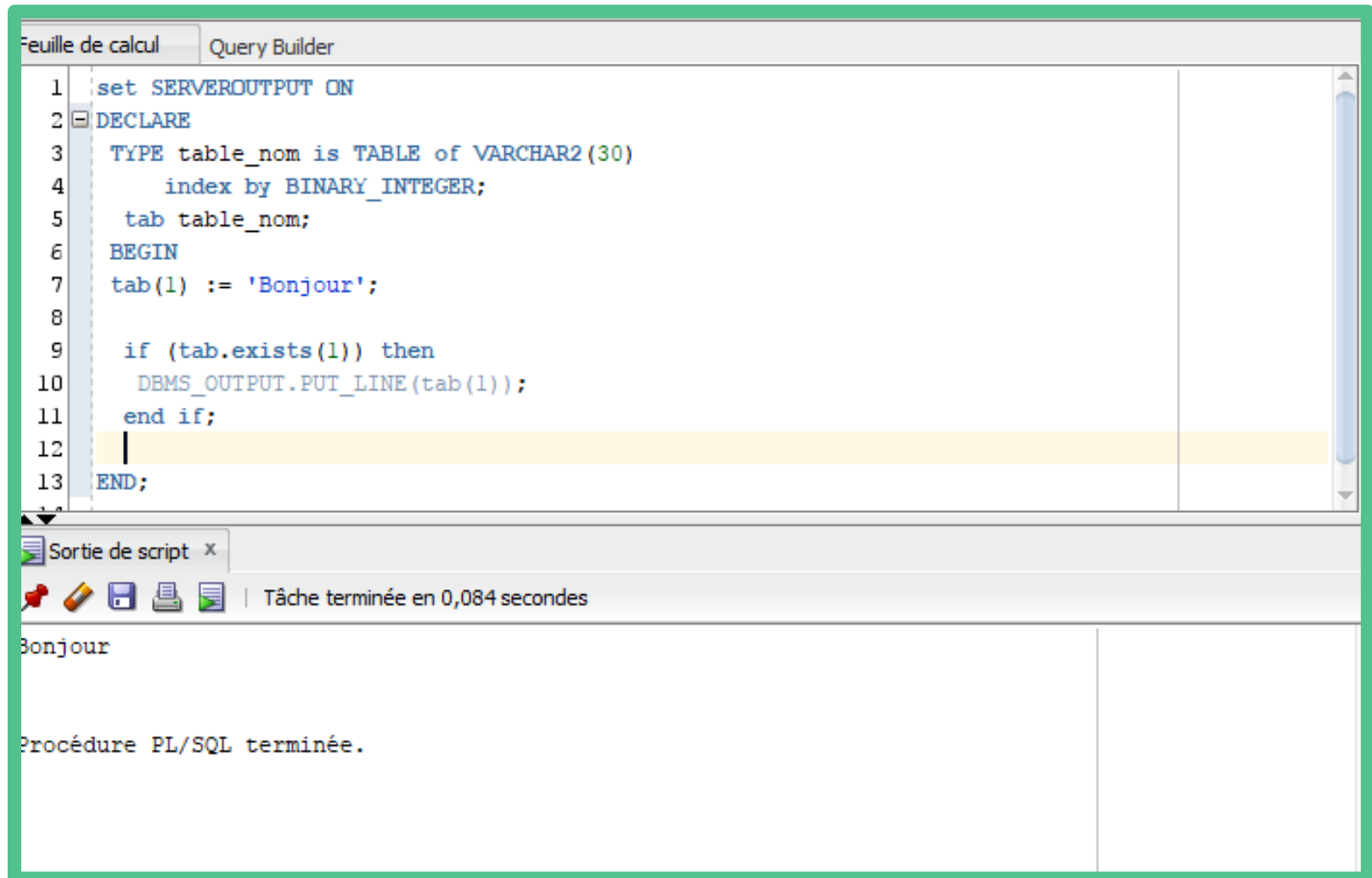
Fonction	
COUNT	retourne le nombre d'éléments de la table
EXISTS (n)	retourne TRUE si l'élément n existe
LAST	retourne la valeur du dernier index
FIRST	retourne le premier index
NEXT (n)	élément suivant
PRIOR(n)	élément précédent
DELETE	supprime toute la table
DELETE(n)	supprime l'élément n

Exemple :

```
IF table_name.EXISTS(1) THEN  
    INSERT INTO ...  
    ...
```

Gestion des tableaux

Exemple 1 :



The screenshot shows a software interface with two main panes. The top pane, titled 'Query Builder', contains a PL/SQL script. The script starts with 'set SERVEROUTPUT ON', followed by a 'DECLARE' section where a table type 'table_nom' is defined with 'VARCHAR2(30)' elements and a 'BINARY_INTEGER' index. A table variable 'tab' of this type is declared. The 'BEGIN' block contains an assignment 'tab(1) := 'Bonjour';', an 'if (tab.exists(1)) then' condition, and a call to 'DBMS_OUTPUT.PUT_LINE(tab(1));'. The script ends with 'END;'. The bottom pane, titled 'Sortie de script', shows the output of the script execution, which is 'Bonjour'. Below the output, a status bar indicates 'Tâche terminée en 0,084 secondes' and 'Procédure PL/SQL terminée.'.

```
1 set SERVEROUTPUT ON
2 DECLARE
3     TYPE table_nom is TABLE of VARCHAR2(30)
4         index by BINARY_INTEGER;
5     tab table_nom;
6 BEGIN
7     tab(1) := 'Bonjour';
8
9     if (tab.exists(1)) then
10         DBMS_OUTPUT.PUT_LINE(tab(1));
11     end if;
12
13 END;
```

Sortie de script x

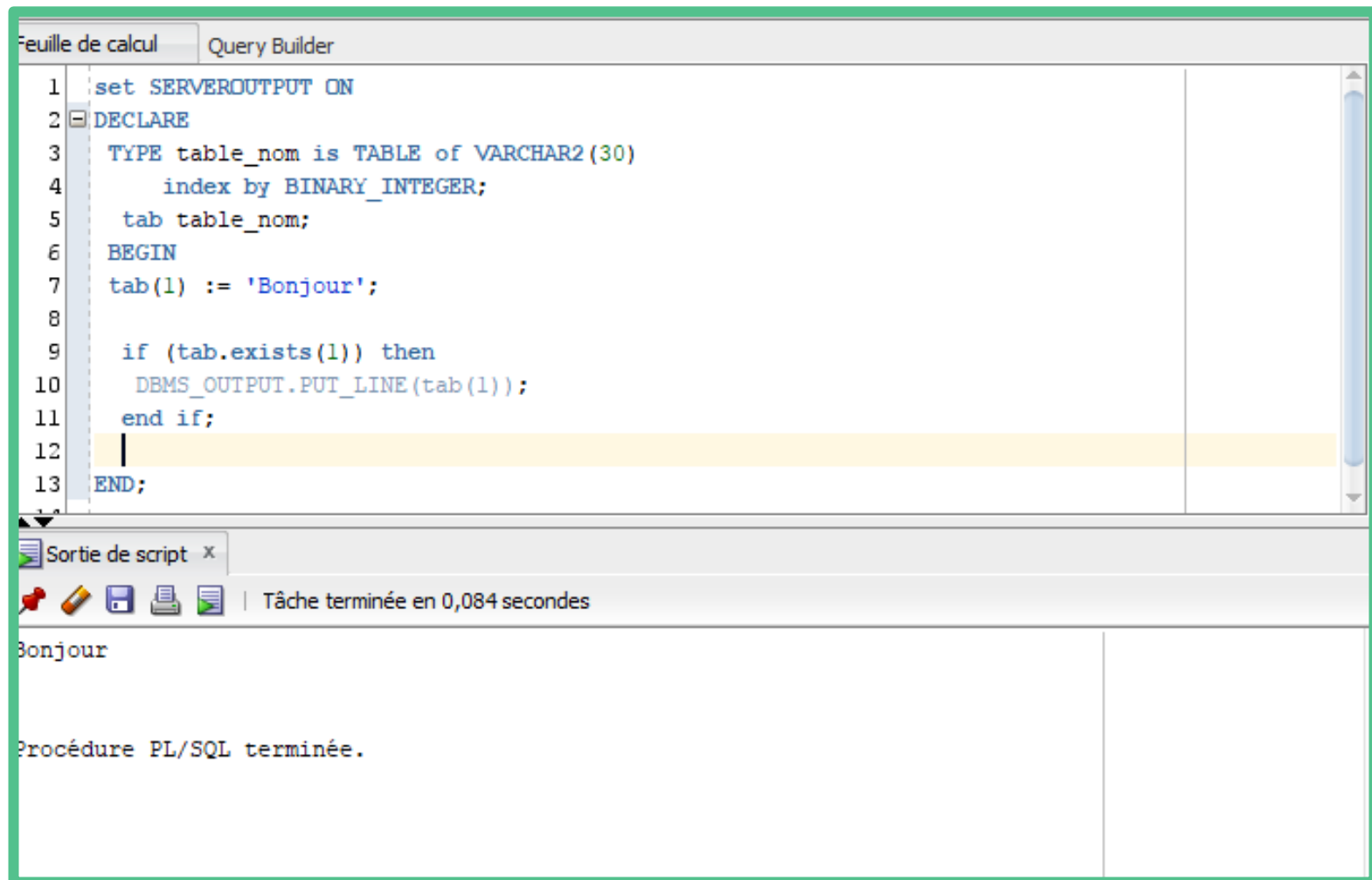
Tâche terminée en 0,084 secondes

Bonjour

Procédure PL/SQL terminée.

Gestion des tableaux

Exemple 2 :



The screenshot shows a software interface with two main panes. The top pane, titled 'Query Builder', contains a PL/SQL script. The script defines a table type, declares a variable, and uses a loop to output the word 'Bonjour'. The bottom pane, titled 'Sortie de script', shows the output of the script, which is 'Bonjour' followed by a blank line and the message 'Procédure PL/SQL terminée.'.

```
1 set SERVEROUTPUT ON
2 DECLARE
3   TYPE table_nom is TABLE of VARCHAR2(30)
4     index by BINARY_INTEGER;
5   tab table_nom;
6 BEGIN
7   tab(1) := 'Bonjour';
8
9   if (tab.exists(1)) then
10     DBMS_OUTPUT.PUT_LINE(tab(1));
11   end if;
12
13 END;
```

Sortie de script x

Tâche terminée en 0,084 secondes

Bonjour

Procédure PL/SQL terminée.

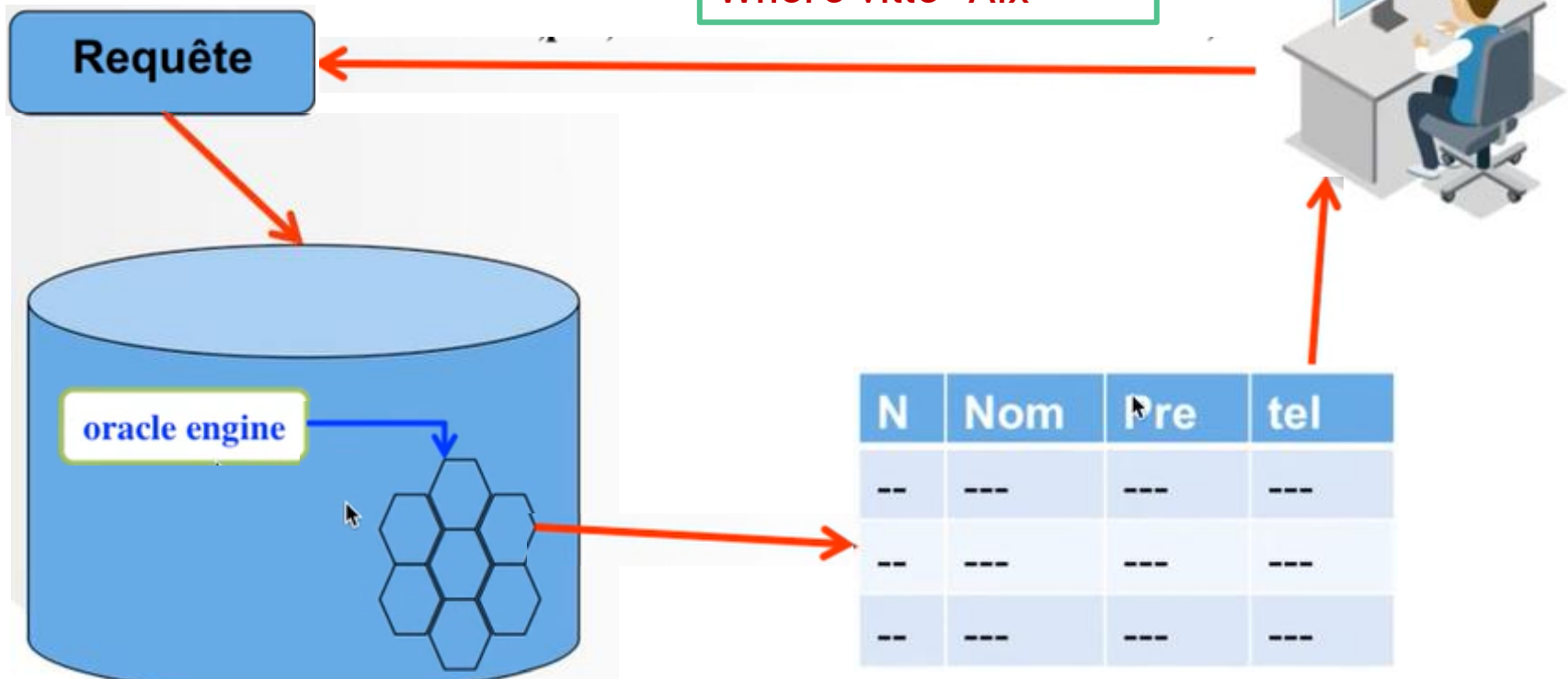
PL/SQL
Les curseurs

Contrôler les curseurs explicites

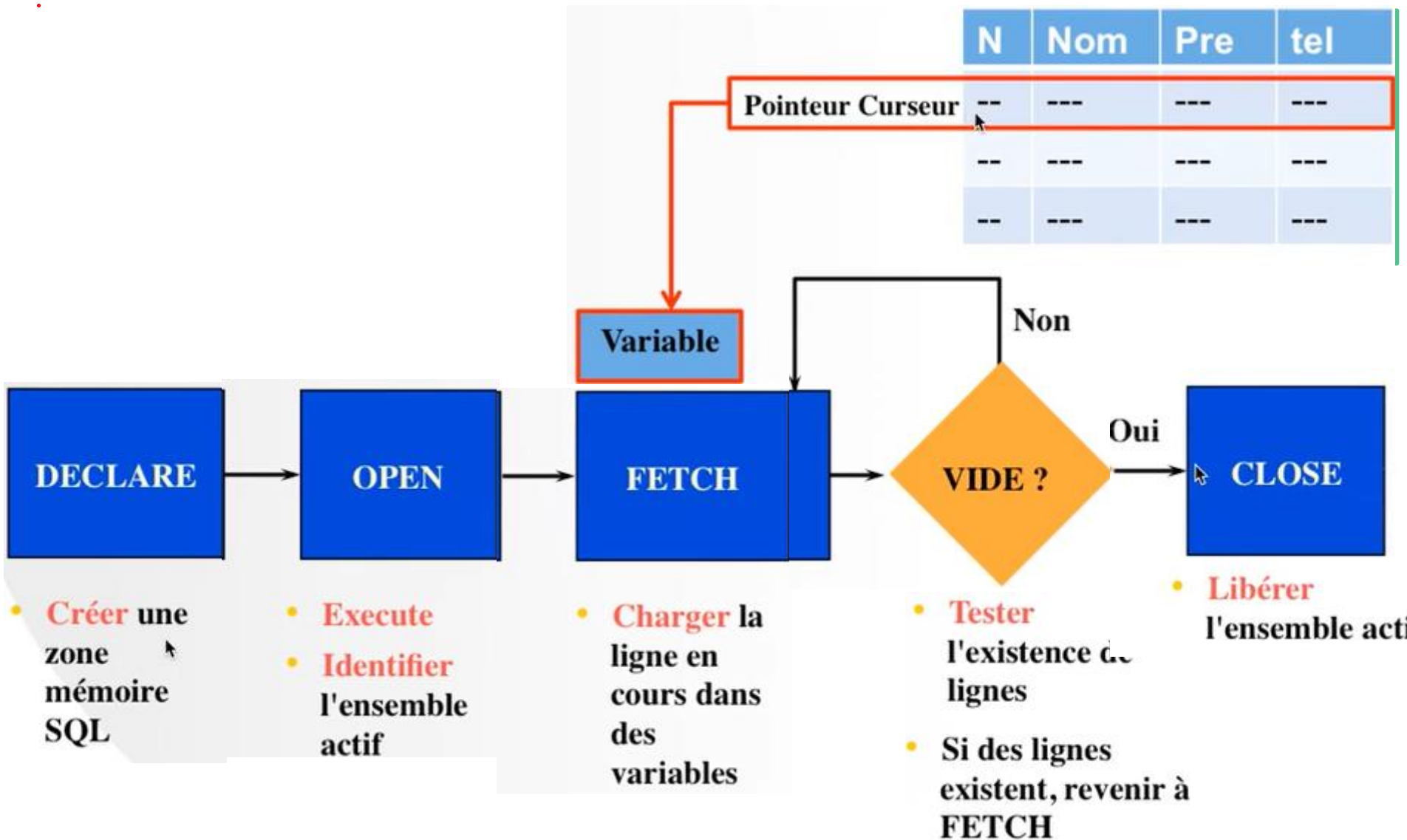
Les curseurs

Un **curseur** est une variable qui pointe vers le **résultat d'une requête SQL**. La déclaration du curseur est liée au texte de la requête.

```
Select nom, pre, tel  
From client  
Where ville='Aix'
```



Contrôler les curseurs explicites



Déclarer le curseur

Syntaxe:

DECLARE

```
CURSOR nom_curseur IS Requête_SQL
```

Exemple :

```
DECLARE
  CURSOR c1 IS
    SELECT employe_id, nom
    FROM   employe;
  CURSOR c2 IS
    SELECT *
    FROM   department
    WHERE  department_id = 10;
BEGIN
  ...
```

Ouvrir le curseur

Syntaxe:

```
OPEN nom_curseur;
```

- ☐ Ouvrir le curseur pour exécuter l'interrogation
- ☐ Si l'interrogation ne renvoie pas de ligne, aucune exception n'est déclenchée
- ☐ Utiliser les attributs du curseur pour tester le résultat après une extraction

Ouvrir le curseur

Syntaxe:

```
CLOSE nom_curseur;
```

- ❑ **Fermer** le curseur après avoir terminé le traitement des lignes
- ❑ **Rouvrir** le curseur, si nécessaire
- ❑ Ne pas essayer d'extraire les données d'un curseur s'il a été fermé

Extraire les données à partir du curseur

Syntaxe:

```
FETCH nom_curseur INTO var_curseur;
```

```
DECLARE
    v_empid  employe.employe_id%TYPE;
    v_nom    employe.nom%TYPE;
CURSOR c1 IS
    SELECT employe_id, nom
    FROM   employe;
BEGIN
    OPEN c1;
    FOR i IN 1..10 LOOP
        FETCH c1 INTO v_empid, v_nom;
        ...
    END LOOP;
    CLOSE c1;
END ;
```

Attributs d'un curseur explicite

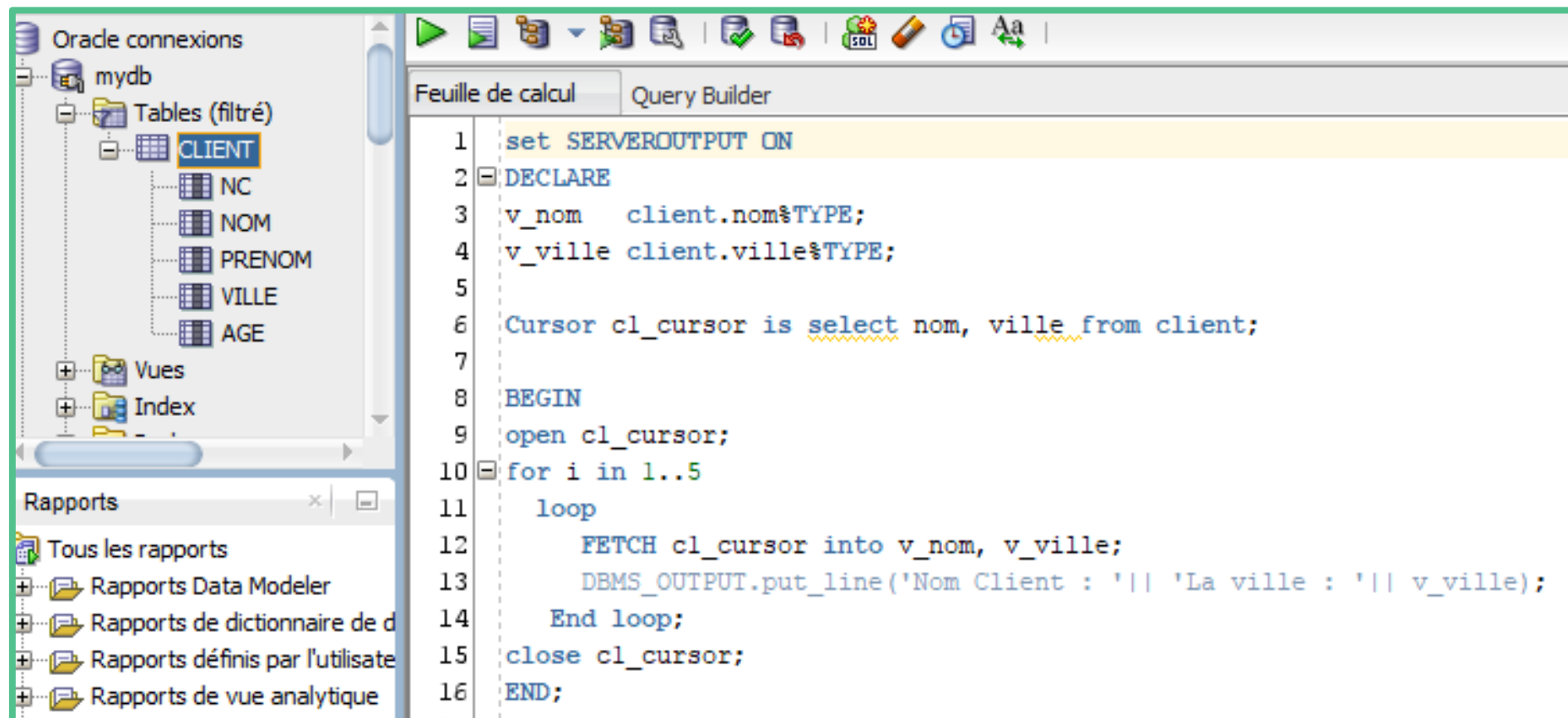
Obtenir les informations d'état concernant un curseur :

`nom_curseur%attribut`

Attribut	Type	Description
<code>%ISOPEN</code>	BOOLEAN	Prend la valeur TRUE si le curseur est ouvert
<code>%NOTFOUND</code>	BOOLEAN	Prend la valeur TRUE si la dernière extraction ne renvoie pas de ligne
<code>%FOUND</code>	BOOLEAN	Prend la valeur TRUE si la dernière extraction renvoie une ligne ; complément de <code>%NOTFOUND</code>
<code>%ROWCOUNT</code>	NUMBER	Prend la valeur correspondant au nombre total de lignes renvoyées jusqu'à présent

Attributs d'un curseur explicite

Boucle FOR pour un curseur



The screenshot displays the Oracle SQL Developer interface. On the left, the 'Orade connexions' tree shows a connection to 'mydb' with a table 'CLIENT' selected. The table's columns are listed: NC, NOM, PRENOM, VILLE, and AGE. Below this, the 'Rapports' section is visible. The main window, titled 'Feuille de calcul' and 'Query Builder', contains a PL/SQL script. The script sets 'SERVEROUTPUT ON', declares a cursor 'cl_cursor' for a query on 'CLIENT', and uses a 'FOR' loop to iterate through the results, printing each row's data.

```
1  set SERVEROUTPUT ON
2  DECLARE
3  v_nom  client.nom%TYPE;
4  v_ville client.ville%TYPE;
5
6  Cursor cl_cursor is select nom, ville from client;
7
8  BEGIN
9  open cl_cursor;
10 for i in 1..5
11     loop
12         FETCH cl_cursor into v_nom, v_ville;
13         DBMS_OUTPUT.put_line('Nom Client : '|| 'La ville : '|| v_ville);
14     End loop;
15 close cl_cursor;
16 END;
```

Attributs d'un curseur explicite

Boucle LOOP pour un curseur

```
1  set SERVEROUTPUT ON
2  DECLARE
3  v_nom    client.nom%TYPE;
4  v_ville  client.ville%TYPE;
5
6  Cursor cl_cursor is select nom, ville from client;
7
8  BEGIN
9  open cl_cursor;
10 Loop
11     FETCH cl_cursor into v_nom, v_ville;
12     exit when cl_cursor%notfound ;
13     DBMS_OUTPUT.put_line('Nom Client : '|| 'La ville : '|| v_ville);
14 End loop;
15 close cl_cursor;
16 END;
```



PL/SQL

Les exceptions

Gestion des erreurs

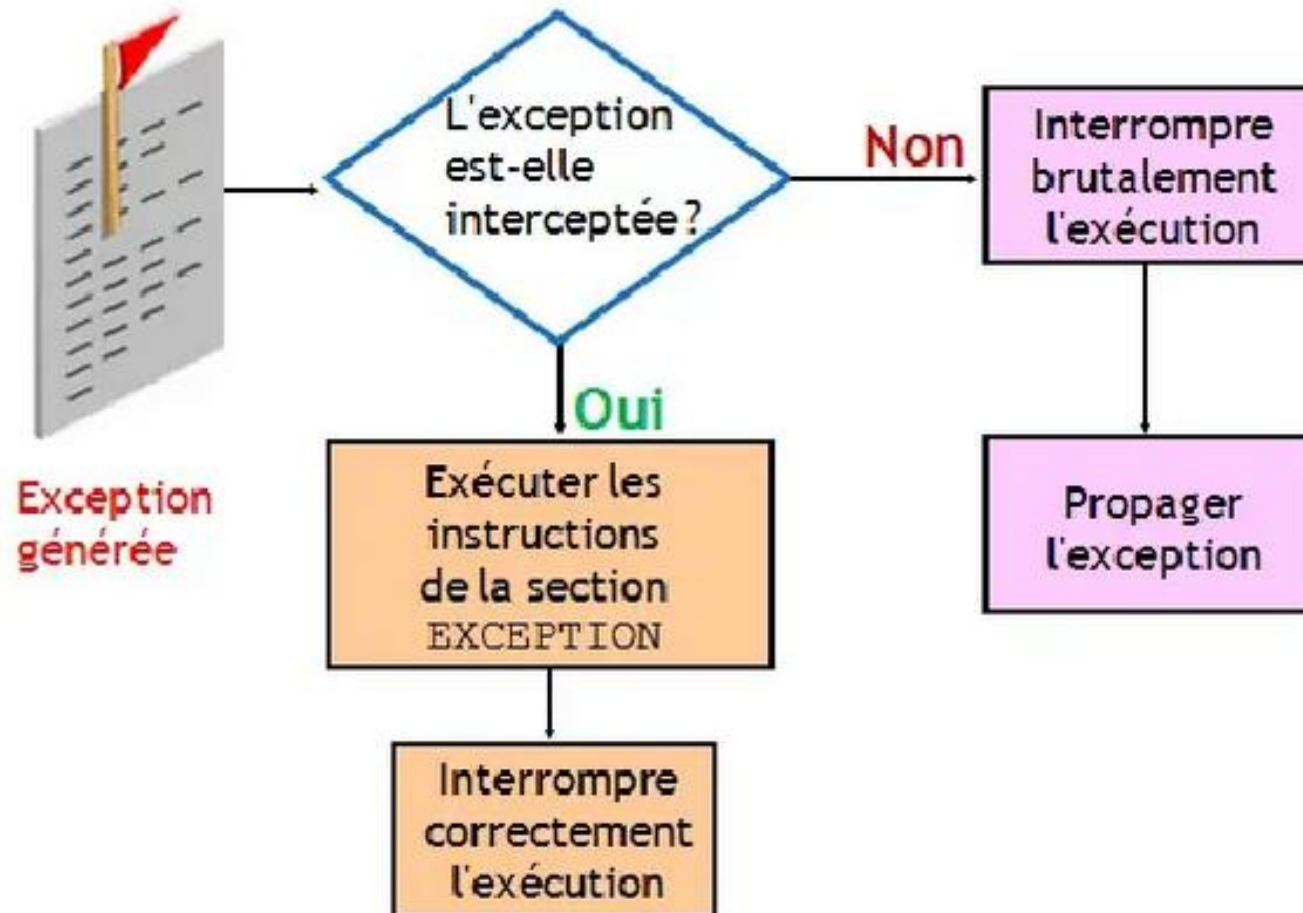


Le langage PL/SQL offre au développeur un mécanisme **de gestion des exceptions**.

Une **exception** est tout **avertissement** (Warning) ou **erreur** détectée lors de la compilation d'un bloc PL/SQL

Il permet de préciser la logique du traitement **des erreurs** survenues dans un bloc PL/SQL?

Gestion des erreurs



Les types d'exceptions



Exception prédéfinie : est une exception définie en interne par PL/SQL et est levée implicitement par le serveur oracle. Par exemple: NO_DATA_FOUND, TOO_MANY_ROWS, etc.

Exception définie par l'utilisateur: Vous pouvez déclarer vos propres exceptions dans la partie déclarative d'un bloc PL/SQL anonyme, sous-programme (procédure, fonction), ou package.

Exception prédéfinie

Les erreurs ORACLE générées par le noyau **sont numérotées** (ORA-xxxx)

Il a donc fallu établir une **table de correspondance** entre les erreurs ORACLE et des noms d'exceptions.

Voici quelques exemples d'exceptions prédéfinis et des codes correspondants:

- Les fonctions d'interception des exceptions:

- **SQLCODE** : renvoie la valeur numérique du code d'erreur
- **SQLERRM** : renvoie le message associé au code d'erreur

Nom d'exception	Erreur ORACLE	SQLCODE
CURSOR_ALREADY_OPEN	ORA-06511	-6511
DUP_VAL_ON_INDEX	ORA-00001	-1
INVALID_CURSOR	ORA-01001	-1001
INVALID_NUMBER	ORA-01722	-1722
LOGIN_DENIED	ORA-01017	-1017
NO_DATA_FOUND	ORA-01403	+100
PROGRAM_ERROR	ORA-06501	-6501
ROWTYPE_MISMATCH	ORA-06504	-6504
TIMEOUT_ON_RESOURCE	ORA-00051	-51
TOO_MANY_ROWS	ORA-01422	-1422
ZERO_DIVIDE	ORA-01476	-1476

Exception prédéfinie

Signification des erreurs Oracles présentées ci-dessus :

- **CURSOR_ALREADY_OPEN** : tentative d'ouverture d'un curseur déjà ouvert.
- **DUP_VAL_ON_INDEX** : violation de l'unicité lors d'une mise à jour détectée au niveau de l'index unique
- **INVALID_CURSOR**: opération incorrecte sur un curseur, comme par exemple la fermeture d'un curseur qui n'a pas été ouvert.
- **INVALID_NUMBER**: échec de la conversion d'une chaîne de caractères en numérique
- **LOGIN_DENIED**: connexion à la base échouée car le nom utilisateur ou le mot de passe est invalide.
- **NO_DATA_FOUND**: déclenché si la commande SELECT INTO ne retourne aucune ligne ou si on fait référence à un enregistrement non initialisé d'un tableau PL/SQL
- **PROGRAM_ERROR**: problème général du au PL/SQL.
- **ROWTYPE_MISMATCH**: survient lorsque une variable curseur d'un programme hôte retourne une valeur dans une variable curseur d'un bloc PL/SQL qui n'a pas le même type.
- **TIMEOUT_ON_RESOURCE**: dépassement du temps dans l'attente de libération des ressources (lié aux paramètres de la table).
- **TOO_MANY_ROWS**: la commande SELECT INTO retourne plus d'une ligne.
- **ZERO_DIVIDE**: tentative de division par zéro

Gestion des erreurs

Le langage PL/SQL offre au développeur un mécanisme **de gestion des exceptions**.

Une exception est tout **avertissement** (Warning) ou **erreur** détectée lors de la compilation d'un bloc PL/SQL

Il permet de préciser la logique du traitement **des erreurs** survenues dans un bloc PL/SQL?

Il s'agit donc d'un point clé dans l'efficacité du langage qui permettra de **protéger l'intégrité du système**.

- **Interne** : Les exceptions internes sont générées par le moteur du système (division par zéro, connexion non établie, table inexistante, privilèges insuffisants, mémoire saturée, espace disque insuffisant,...)
- **Externe** : Les exceptions externes sont générées par l'utilisateur.

Gestion des erreurs

Les erreurs ORACLE générées par le noyau **sont numérotées** (ORA-xxxxx)

Il a donc fallu établir une **table de correspondance** entre les erreurs ORACLE et des noms d'exceptions.

Voici quelques exemples **d'exceptions prédéfinis** et des codes correspondants:

Nom d'exception	Erreur ORACLE	SQLCODE
CURSOR_ALREADY_OPEN	ORA-06511	-6511
DUP_VAL_ON_INDEX	ORA-00001	-1
INVALID_CURSOR	ORA-01001	-1001
INVALID_NUMBER	ORA-01722	-1722
LOGIN_DENIED	ORA-01017	-1017
NO_DATA_FOUND	ORA-01403	+100
PROGRAM_ERROR	ORA-06501	-6501
ROWTYPE_MISMATCH	ORA-06504	-6504
TIMEOUT_ON_RESOURCE	ORA-00051	-51
TOO_MANY_ROWS	ORA-01422	-1422
ZERO_DIVIDE	ORA-01476	-1476

Gestion des erreurs

Signification des erreurs Oracles présentées ci-dessus :

- **CURSOR_ALREADY_OPEN** : tentative d'ouverture d'un curseur déjà ouvert.
- **DUP_VAL_ON_INDEX** : violation de l'unicité lors d'une mise à jour détectée au niveau de l'index unique
- **INVALID_CURSOR**: opération incorrecte sur un curseur, comme par exemple la fermeture d'un curseur qui n'a pas été ouvert.
- **INVALID_NUMBER**: échec de la conversion d'une chaîne de caractères en numérique
- **LOGIN_DENIED**: connexion à la base échouée car le nom utilisateur ou le mot de passe est invalide.
- **NO_DATA_FOUND**: déclenché si la commande SELECT INTO ne retourne aucune ligne ou si on fait référence à un enregistrement non initialisé d'un tableau PL/SQL
- **PROGRAM_ERROR**: problème général du au PL/SQL.
- **ROWTYPE_MISMATCH**: survient lorsque une variable curseur d'un programme hôte retourne une valeur dans une variable curseur d'un bloc PL/SQL qui n'a pas le même type.
- **TIMEOUT_ON_RESOURCE**: dépassement du temps dans l'attente de libération des ressources (lié aux paramètres de la table).
- **TOO_MANY_ROWS**: la commande SELECT INTO retourne plus d'une ligne.
- **ZERO_DIVIDE**: tentative de division par zéro

Exception utilisateur

Syntaxe:

[DECLARE]

nom_erreur EXCEPTION;

1

BEGIN

IF (anomalie) THEN
RAISE nom_erreur;
END IF;

2

[EXCEPTION]

WHEN nom_erreur THEN traitement;

3

END;

Exception utilisateur

Exemple 1 : Exception utilisateur

```
DECLARE
  Employe_age INT;
  invalid_age EXCEPTION;
BEGIN
  SELECT age INTO Employe_age FROM Employe WHERE nom='Salami';

  IF Employe_age <= 0 THEN
    RAISE invalid_age;
  ELSE
    DBMS_OUTPUT.PUT_LINE('L age de l'employé est: '||Employe_age);
  END IF;

  EXCEPTION
  WHEN invalid_age THEN dbms_output.put_line('L age est invalide');

END;
```

Exception utilisateur

Exemple 2 : Ecrire un programme PL/SQL qui permet de lire deux valeurs entières A et B et qui affiche le résultat de la division de A par B. Gérer le cas où B est égal à zéro sous forme d'une exception

```
SET SERVEROUTPUT ON
SET VERIFY OFF
Prompt donner la valeur de A:
Accept a
Prompt Donner la valeur de B:
Accept B
Declare
  c number;
  division_zero Exception;
Begin
  If &b<> 0 then
    c:=&a/&b;
    DBMS_OUTPUT.PUT_LINE ('Le résultat = '||c);
  else
    raise division_zero;
  end If;
Exception
  When division_zero then
    DBMS_OUTPUT.PUT_LINE('!!!! Division Impossible !!!!!');
End;
/
```

Exception Oracle - Prédéfinie

Syntaxe

```
BEGIN  SELECT ... COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    statement1;
    statement2;
  WHEN TOO_MANY_ROWS THEN
    statement1;
  WHEN OTHERS THEN
    statement1;
    statement2;
    statement3;
END;
```

Exception Oracle – Prédéfinie

Exemple 1 : Exception prédéfinie Oracle

```
DECLARE
Var_Nom EMPLOYE.nom%TYPE;
BEGIN
SELECT nom INTO Var_Nom FROM EMPLOYE WHERE numéro=1;

EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('L'employé n'existe pas') ;

WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Erreur' ) ;
END;
/
```

Exception Oracle – Prédéfinie

Exemple 2 : Exception prédéfinie Oracle

DECLARE

Iname VARCHAR2(15);

BEGIN

SELECT last_name INTO Iname FROM employees

WHERE first_name='John';

DBMS_OUTPUT.PUT_LINE ('Le nom de John est : ' || Iname);

EXCEPTION

WHEN TOO_MANY_ROWS THEN

DBMS_OUTPUT.PUT_LINE ('La requête revoie plusieurs lignes, utilisez un curseur');

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE ('Aucun employé ne porte le prénom John');

END;

Exception Oracle – Non Prédéfinie

Dans ce type nous créons une **correspondance** entre le code erreur ORACLE et le nom de l'**exception**.

Ce nom est choisi librement par le programmeur.

Pour affecter un nom à un code erreur, on doit utiliser une directive compilée (pragma) nommée **EXCEPTION_INIT**.

Syntaxe

[DECLARE]

```
champ_obligatoire EXCEPTION;
```

1

```
PRAGMA EXCEPTION_INIT (champ_obligatoire, -1400);
```

2

BEGIN...

```
EXCEPTION
```

```
    WHEN champ_obligatoire THEN traitement;
```

3

END;

Redéclaration des exceptions prédéfinies

Dans ce type nous créons une correspondance entre le **code erreur ORACLE** et le **nom de l'exception**.

Ce nom est choisi librement par le programmeur.

Pour affecter un nom à **un code erreur**, on doit utiliser une directive compilée (**pragma**) nommée **EXCEPTION_INIT**.

Syntaxe

[DECLARE]

```
champ_obligatoire EXCEPTION;
```

1

```
PRAGMA EXCEPTION_INIT (champ_obligatoire, -1400);
```

2

```
EXCEPTION  
  WHEN champ_obligatoire THEN traitement;
```

3

END;

Redéclaration des exceptions prédéfinies

Exemple 1 : Exception Orale – non prédéfinie

```
DECLARE
    a number:=100;
    b number:=0;
    division_zero EXCEPTION;
    PRAGMA EXCEPTION_INIT(division_zero,-01476);
BEGIN
    a:=a/b;
    EXCEPTION
        WHEN division_zero THEN
            DBMS_OUTPUT.PUT_LINE('ERREUR: Division par 0');
END;
```

Le code erreur -100 correspond aux données inexistantes .

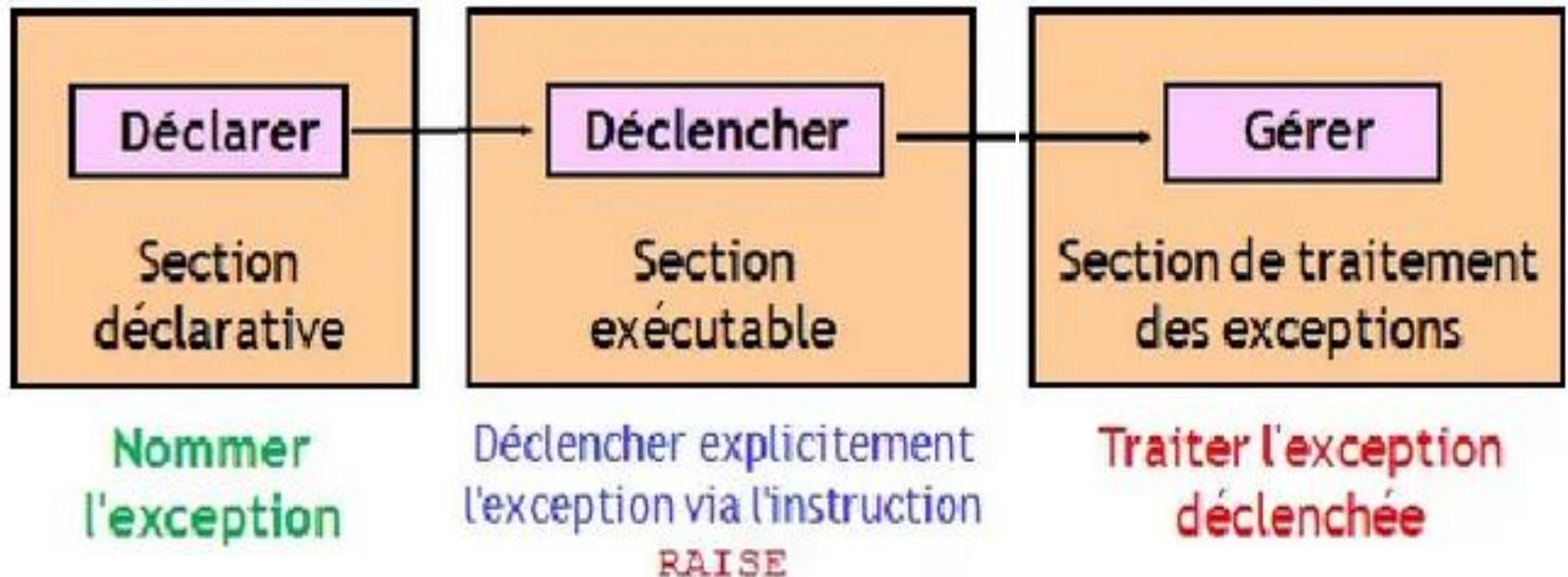
Redéclaration des exceptions prédéfinies

Exemple 2 : Exception Orale – non prédéfinie

```
DECLARE  
Erreur_data EXCEPTION;  
Var_Nom EMPLOYE.nom%TYPE;  
PRAGMA EXCEPTION_INIT (Erreur_data , -100);  
BEGIN  
SELECT nom INTO var_nom FROM Employé WHERE code=1;  
EXCEPTION  
    WHEN Erreur_data THEN  
    DBMS_OUTPUT.PUT_LINE('L'employé n'existe pas' ) ;  
END;
```

Le code erreur -100 correspond aux données inexistantes .

Déclencher des exceptions explicitement



Déclencher des exceptions explicitement

Avec le mot clé RAISE

Syntaxe

DECLARE

exception_name EXCEPTION; – déclarer l'exception
[PRAGMA EXCEPTION_INIT(exception_name,error_code)];

...

BEGIN

...

RAISE exception_name; – déclencher l'exception

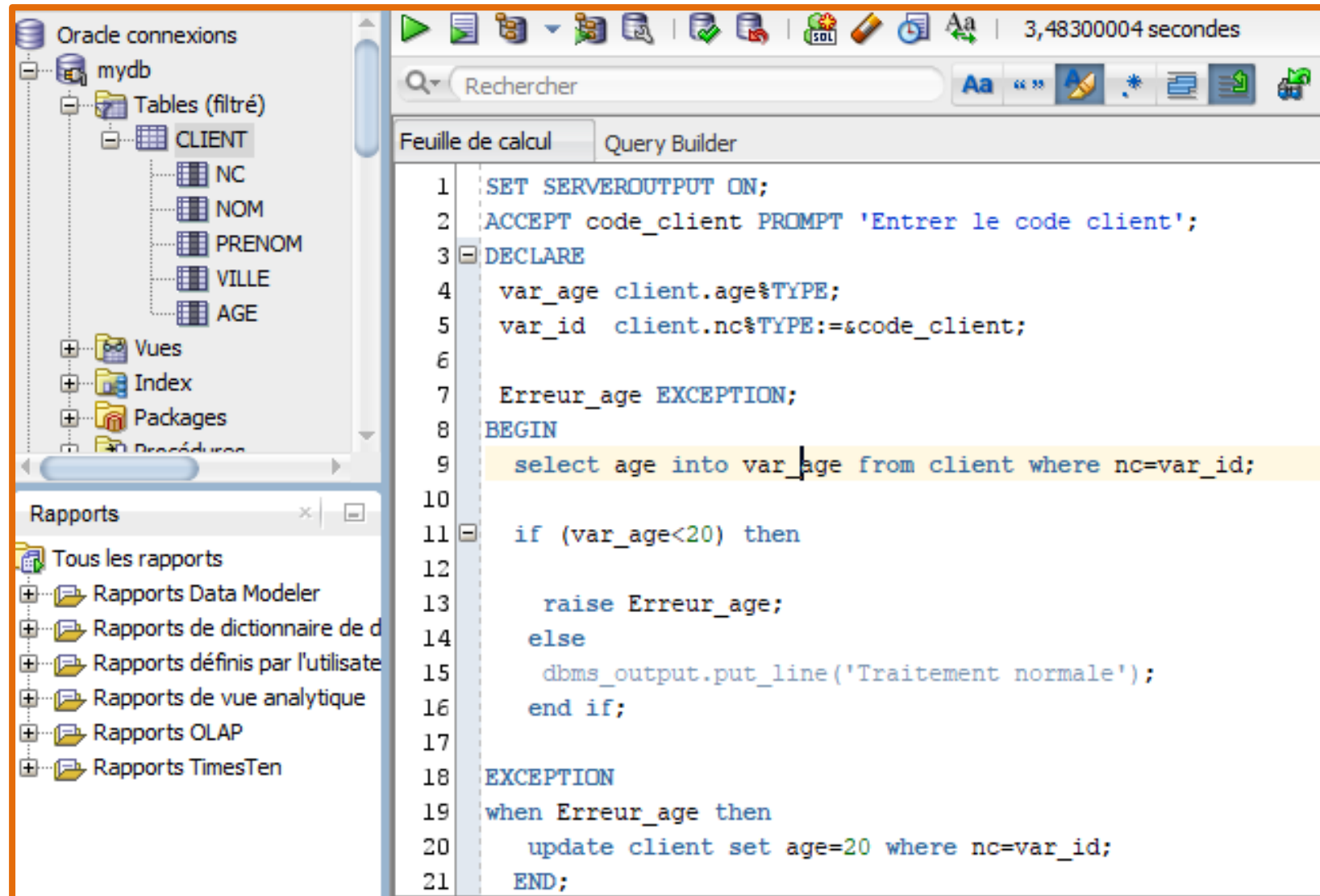
EXCEPTION

WHEN exception_name THEN – gérer l'exception
– traitement;

...

END;

Exemple : Exception utilisateur



The screenshot displays the Oracle SQL Developer interface. On the left, the 'Orade connexions' tree shows a connection to 'mydb'. Under 'Tables (filtré)', the 'CLIENT' table is selected, showing columns: NC, NOM, PRENOM, VILLE, and AGE. Below this, the 'Rapports' section lists various report types, including 'Tous les rapports', 'Rapports Data Modeler', 'Rapports de dictionnaire de données', 'Rapports définis par l'utilisateur', 'Rapports de vue analytique', 'Rapports OLAP', and 'Rapports TimesTen'.

The main window shows the 'Query Builder' tab with a PL/SQL script. The script is as follows:

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrer le code client';
3 DECLARE
4   var_age client.age%TYPE;
5   var_id  client.nc%TYPE:=&code_client;
6
7   Erreur_age EXCEPTION;
8 BEGIN
9   select age into var_age from client where nc=var_id;
10
11   if (var_age<20) then
12
13     raise Erreur_age;
14   else
15     dbms_output.put_line('Traitement normale');
16   end if;
17
18 EXCEPTION
19 when Erreur_age then
20   update client set age=20 where nc=var_id;
21 END;
```

The script uses an exception handler to manage a user-defined exception 'Erreur_age'. It prompts the user for a client code, attempts to select the age of the client with that code, and if the age is less than 20, it raises the 'Erreur_age' exception. The exception handler then updates the client's age to 20.

Exemple 2: Exception Oracle - Prédéfinie

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Oracle connexions' tree shows a connection to 'mydb' with a schema containing tables like 'CLIENT' (with columns NC, NOM, PRENOM, VILLE, AGE), 'Vues', 'Index', 'Packages', and 'Procédures'. Below this is a 'Rapports' (Reports) section. The main window is titled 'Feuille de calcul' and 'Query Builder', showing a PL/SQL script with the following code:

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrer le code client';
3 DECLARE
4   var_age client.age%TYPE;
5   var_id  client.nc%TYPE:=&code_client;
6
7   Erreur_age EXCEPTION;
8 BEGIN
9   dbms_output.put_line('1/Numero client' || 1/var_id);
10  select age into var_age from client where nc=var_id;
11
12 EXCEPTION
13 when NO_DATA_FOUND then
14   dbms_output.put_line('Ce client n'existe pas dans la base de données');
15 when ZERO_DIVIDE then
16   dbms_output.put_line('Tentative de division par 0');
17 When OTHERS then
18   dbms_output.put_line('Erreur inconnue');
19
20 END;
```

The script is designed to accept a client code, attempt a division by its value, and handle two specific predefined exceptions: `NO_DATA_FOUND` (if the client does not exist) and `ZERO_DIVIDE` (if the client code is zero, causing a division by zero). The `ZERO_DIVIDE` exception handling block is highlighted with an orange rectangle in the image.

Exemple 2: Exception Oracle - Prédéfinie

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Oracle connexions' tree shows a connection to 'mydb' with a schema containing tables like CLIENT, NC, NOM, PRENOM, VILLE, and AGE. The main window is the 'Query Builder' showing a PL/SQL script. A dialog box 'Entrer une valeur' is open, prompting for 'code client' with the value '0' entered. The script includes a 'when ZERO_DIVIDE then' block, which is highlighted with an orange rectangle. Below the script, the 'Sortie de script' window shows the output 'Tentative de division par 0', which is highlighted with a red rectangle.

Oracle connexions
mydb
Tables (filtré)
CLIENT
NC
NOM
PRENOM
VILLE
AGE
Vues
Index
Packages
Procédures

Rapports
Tous les rapports
Rapports Data Modeler
Rapports de dictionnaire de données
Rapports définis par l'utilisateur
Rapports de vue analytique
Rapports OLAP
Rapports TimesTen

Feuille de calcul | Query Builder

3,20799994 secondes

```
1 SET SERVEROUTPUT ON;  
2 ACCEPT code_client PROMPT 'Entrer le code client';  
3 DECLARE  
4   var_age client.age%TYPE;  
5   var_id client.nc%TYPE:=code_client;  
6  
7   Erreur_e  
8 BEGIN  
9   dbms_output.put_line('code client: ' || 1/var_id);  
10  select  
11  ere nc=var_id;  
12 EXCEPTION  
13 when NO_DATA_FOUND then  
14   dbms_output.put_line('Ce client n''existe pas dans la base de données');  
15 when ZERO_DIVIDE then  
16   dbms_output.put_line('Tentative de division par 0');  
17 When OTHERS then
```

Entrer une valeur

Entrer le code client

0

OK Annuler

Sortie de script

Tâche terminée en 90,684 secondes

END;
Tentative de division par 0

Exemple 3: Exception Oracle – Prédéfinie

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Orade connexions' tree shows a connection to 'mydb' with a schema containing tables 'CLIENT', 'NC', 'NOM', 'PRENOM', 'VILLE', and 'AGE'. The main window shows a PL/SQL script in the 'Feuille de calcul' tab. The script includes a 'SET SERVEROUTPUT ON;' statement, a prompt for a client code, and a query that selects client details based on the input code. It also features exception handling for 'NO_DATA_FOUND' and 'ZERO_DIVIDE'. A dialog box prompts the user to enter a value for 'code client', with '10' entered. The 'Sortie de script' window shows the execution results, including the error message 'Ce client n'existe pas dans la base de données'.

	NC	NOM	PRENOM	VILLE	AGE
1	5	Léa	Morel	Paris	32
2	3	Camille	Durand	Marseille	34
3	1	Emlias	Fontaine	Lille	60
4	4	Manon	Roussel	Amiens	40
5	2	Nolan	Perrin	Aix	20

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrer le code client';
3 DECLARE
4   var_age_client age%TYPE;
5   var_id client.nc%TYPE;
6
7   Erreur EXCEPTION;
8 BEGIN
9   dbms_output.put_line('Entrer le code client');
10  select var_id, var_age_client into var_id, var_age_client
11  from client
12  where nc=var_id;
13
14  EXCEPTION
15  when NO_DATA_FOUND then
16    dbms_output.put_line('Ce client n'existe pas dans la base de données');
17  when ZERO_DIVIDE then
18    dbms_output.put_line('Tentative de division par 0');
19  When OTHERS then
20    dbms_output.put_line('Erreur inconnue');
```

Sortie de script x

Tâche terminée en 95,457 secondes

1/Numero client,1
Ce client n'existe pas dans la base de données

Exemple 3: Exception 0

	NC	NOM	PRENOM	VILLE	AGE
1	5	Léa	Morel	Paris	32
2	3	Camille	Durand	Marseille	34
3	1	Emlias	Fontaine	Lille	60
4	4	Manon	Roussel	Amiens	40
5	2	Nolan	Perrin	Aix	20

The screenshot displays the SQL Developer environment. On the left, the 'Connexions' pane shows a tree view of the 'mydb' schema, including tables (CLIENT, NC, NOM, PRENOM, VILLE, AGE), views, indexes, packages, procedures, and functions. The 'Rapports' pane lists various report types. The main window shows a SQL script titled 'Exception-orade-predefinie.sql' in the 'Feuille de calcul SQL' editor. The script is as follows:

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrez le code client';
3 DECLARE
4   var_age client.age%TYPE;
5   var_id client.nc%TYPE:=&code_client;
6
7   Erreur_age EXCEPTION;
8 BEGIN
9   dbms_output.put_line('1/Numero client' || 1/var_id);
10  select age into var_age from client where nc=var_id;
11  insert into client values (1,'aaa','aaa', 'aaa',12);
12
13 EXCEPTION
14 when NO_DATA_FOUND then
15   dbms_output.put_line('Ce client n'existe pas dans la base de données');
16 when ZERO_DIVIDE then
17   dbms_output.put_line('Tentative de division par 0');
18 When OTHERS then
19   dbms_output.put_line('Erreur inconnue');
20
21 END;
```

A dialog box titled 'Entrez une valeur' is open, prompting the user to 'Entrez le code client'. The input field contains the value '1'. The 'OK' button is highlighted.

The 'Sortie de script' window shows the execution result of the script. It indicates that the task was completed in 3.47 seconds. The output is as follows:

```
END;
1/Numero client1
Erreur inconnue
```

Exception Oracle – Non Prédéfinie

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connexions' pane shows database connections (mydb, ORCL, ORCL1). The main window shows a SQL script in the 'Feuille de calcul SQL' editor. The script defines variables for client age and ID, declares exceptions, and includes a PL/SQL block with an exception handler. A dialog box prompts for a client ID, and a table of client data is shown in the top right.

NC	NOM	PRENOM	VILLE	AGE
1	5 Léa	Morel	Paris	32
2	3 Camille	Durand	Marseille	34
3	1 Emlias	Fontaine	Lille	60
4	4 Manon	Roussel	Amiens	40
5	2 Nolan	Perrin	Aix	20

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrer le code client';
3 DECLARE
4   var_age client.age%TYPE;
5   var_id  client.nc%TYPE:=&code_client;
6   Erreur_data      EXCEPTION;
7   plusieurs_ligne  EXCEPTION;
8
9   PRAGMA EXCEPTION_INIT (erreur_data, 100);
10  PRAGMA EXCEPTION_INIT (plusieurs_ligne, -1422);
11
12 BEGIN
13
14   select age into var_age from client where nc=var_id;
15
16 EXCEPTION
17 when Erreur_data then
18   dbms_output.put_line('Ce client n'existe pas dans la base de données');
19 when plusieurs_ligne then
20   dbms_output.put_line('plusieurs client sélectionnés');
21 When OTHERS then
22   dbms_output.put_line('Erreur inconnue');
23
24 END;
```

Sortie de script x

Tâche terminée en 2,444 secondes

END;

Ce client n'existe pas dans la base de données

Exception Oracle – Non Prédéfinie

The screenshot displays the Oracle SQL Developer environment. On the left, the 'Oracle connexions' tree shows a connection to 'mydb' with a schema containing tables 'CLIENT', 'NC', 'NOM', 'PRENOM', 'VILLE', and 'AGE'. The 'Rapports' pane on the bottom left lists various report types. The main 'Query Builder' window shows a PL/SQL script with the following code:

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrer le code client';
3 DECLARE
4   var_age client.age%TYPE;
5   var_id  client.nc%TYPE:=&code_client;
6   Erreur_data      EXCEPTION;
7   plusieurs_ligne  EXCEPTION;
8
9   PRAGMA EXCEPTION_INIT (erreur_data, -100);
10  PRAGMA EXCEPTION_INIT (plusieurs_ligne, -1422);
11
12 BEGIN
13
14   select age into var_age from client;
15
16 EXCEPTION
17   when Erreur_data then
18     dbms_output.put_line('Ce client n'existe pas dans la base de données');
19   when plusieurs_ligne then
20     dbms_output.put_line('plusieurs client sélectionnés');
21   when OTHERS then
22     dbms_output.put_line('Erreur inconnue');
23
24 END;
```

The script is designed to accept a client code and retrieve their age. It uses two custom exceptions: 'Erreur_data' for invalid client codes and 'plusieurs_ligne' for multiple rows returned. The table 'CLIENT' contains the following data:

NC	NOM	PRENOM	VILLE	AGE
1	5 Léa	Morel	Paris	32
2	3 Camille	Durand	Marseille	34
3	1 Emlias	Fontaine	Lille	60
4	4 Manon	Roussel	Amiens	40
5	2 Nolan	Perrin	Aix	20

The 'Sortie de script' window at the bottom shows the execution results: 'Tâche terminée en 3,829 secondes' and the output 'plusieurs client sélectionnés', indicating that the script successfully handled the custom exception for multiple rows.



PL/SQL

Les procédures

Objectifs

- Définition d'une procédure
- Créer une procédure
- Faire la distinction entre les **paramètres formels** et les **paramètres réels**
- Répertorier les fonctions des différents **modes des paramètres**
- Créer des procédures avec des paramètres
- Appeler une procédure
- Traiter des exceptions dans les procédures
- Supprimer une procédure

Définition d'une procédure

- Une **procédure** est un type de sous-programme qui exécute une action
- Une procédure **peut être stockée en tant qu'objet de schéma** dans la base de données en vue d'exécutions répétées.

Les procédures

Pourquoi les procédures ?

- **Réduire le trafic sur le réseau** (les procédures sont locales sur le serveur)
- Masquer la **complexité du code SQL** (simple appel de procédure avec passage d'arguments)
- Mieux garantir **l'intégrité des données** (encapsulation des données par les procédures)
- **Sécuriser l'accès aux données** (accès à certaines tables seulement à travers les procédures)
- **Optimiser le code** (les procédures sont compilées avant l'exécution du programme et elles sont exécutées immédiatement si elles se trouvent dans la SGA (zone mémoire gérée par ORACLE). De plus une procédure peut être exécutée par plusieurs utilisateurs)

Syntaxe pour la création de procédures

```
CREATE [OR REPLACE] PROCEDURE procedure_name
  [(parameter1 [mode1] datatype1,
   parameter2 [mode2] datatype2,
   . . .)]
IS
PL/SQL Block;
```

- L'option **REPLACE** indique que, si la procédure existe, elle sera supprimée et remplacée par la nouvelle version créée avec l'instruction
- Le bloc PL/SQL commence par **BEGIN** ou par la **déclaration** de variables locales et se termine par **END** ou par **END *procedure_name***

Paramètres Formels / Réels

- Les paramètres **formels** sont des variables **déclarées** dans la liste de paramètres d'une spécification de sous-programme.

Exemple

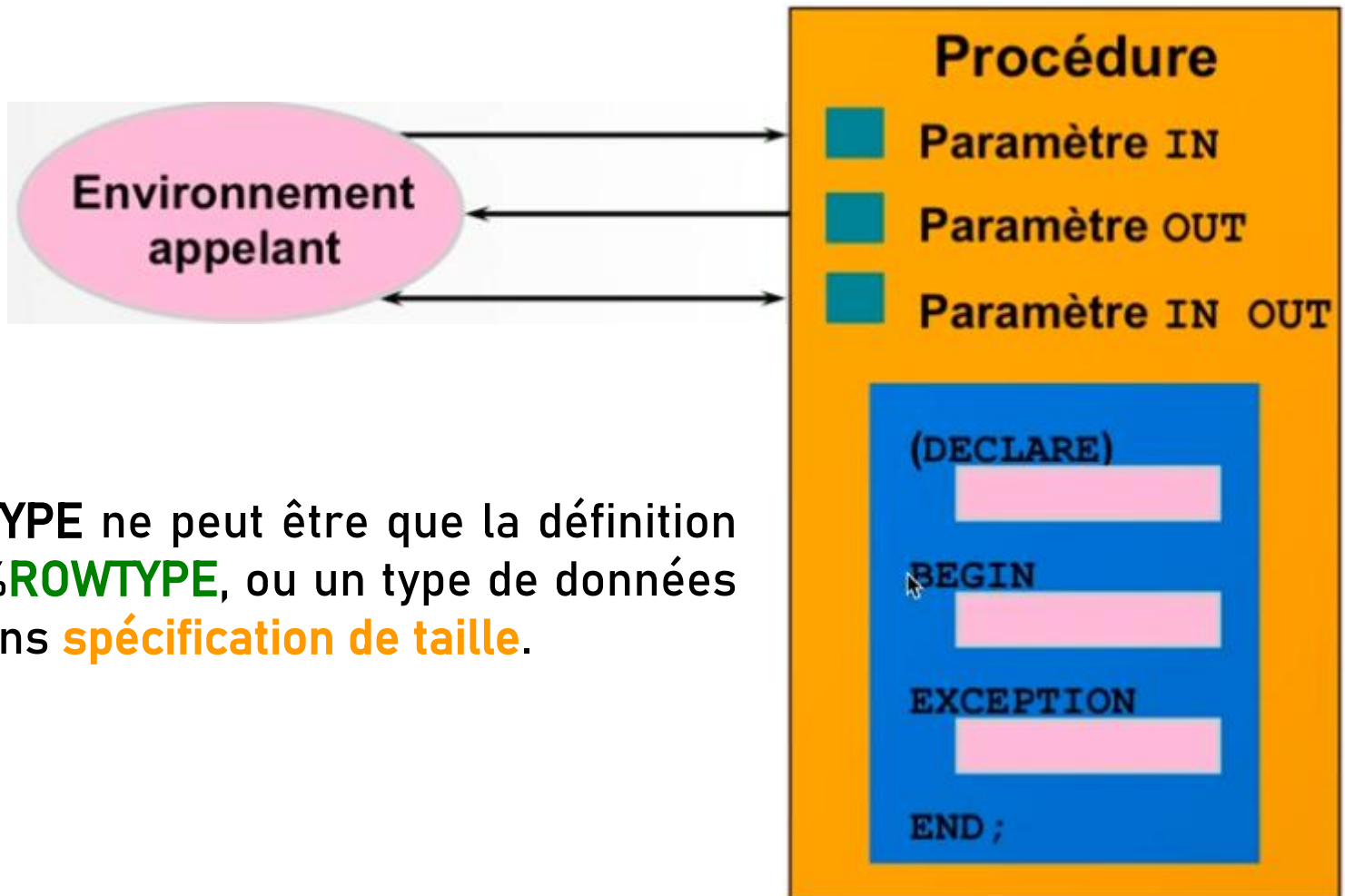
```
CREATE PROCEDURE Moy_sal ( Emp_id NUMBER, Emp_sal NUMBER)
....
END Moy_sal;
```

- Les paramètres réels sont **des variables ou des expressions référencées** dans la liste de paramètres d'un appel de sous-programme

Exemple

```
Moy_sal(v_id, 2000)
```

Modes des paramètres des procédures



RQ : **DATATYPE** ne peut être que la définition **%TYPE** ou **%ROWTYPE**, ou un type de données **explicite** sans **spécification de taille**.

Créer des procédures avec des paramètres

IN	OUT	IN OUT
Mode par défaut	Doit être indiqué	Doit être indiqué
La valeur est transmise au sous-programme	Est renvoyé à l'environnement appelant	Est transmis à un sous-programme ; est renvoyé à l'environnement appelant
Le paramètre formel se comporte en constante	Variable non initialisée	Variable initialisée
Le paramètre réel peut être un littéral, une expression, une constante ou une variable initialisée	Doit être une variable	Doit être une variable

Par défaut, le paramètre **IN** est transmis **par référence** et les paramètres **OUT** et **IN OUT** sont transmis **par valeur**.

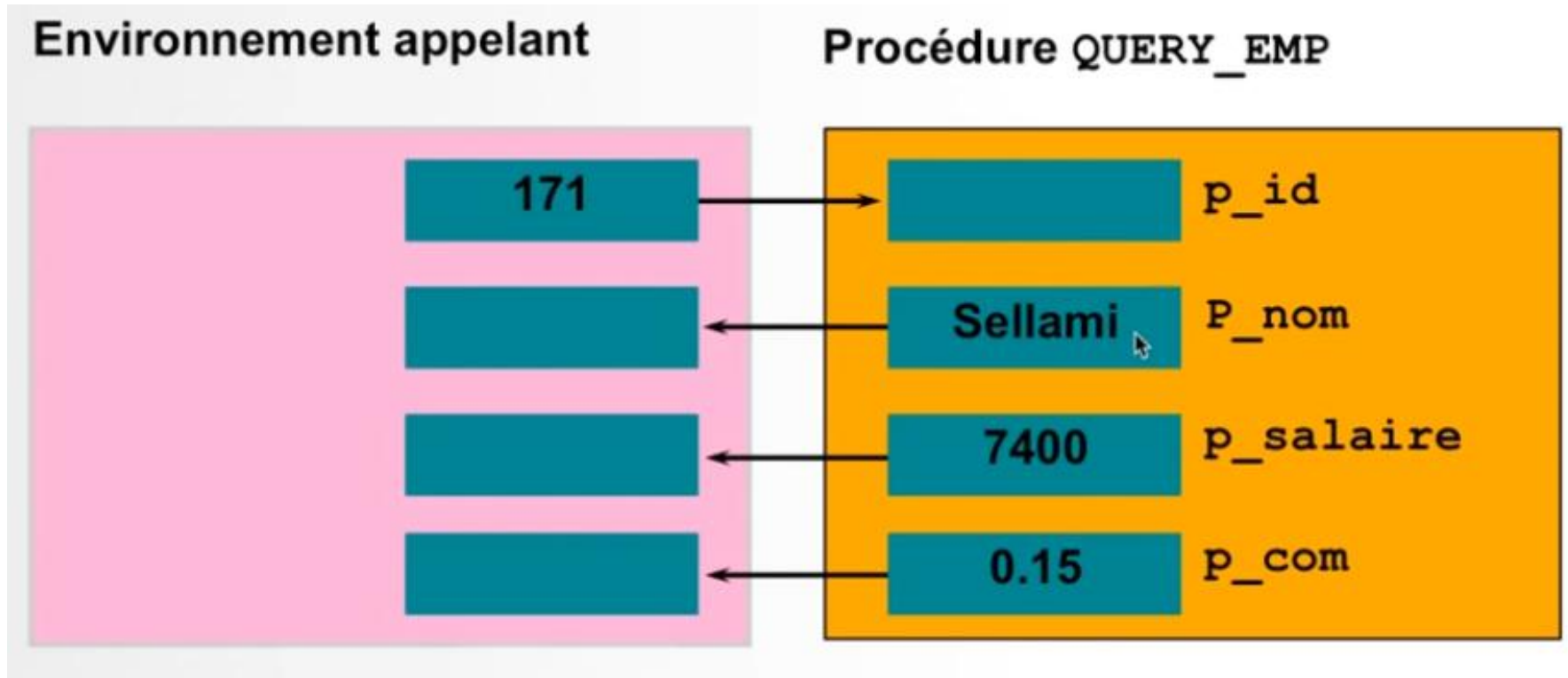
Les procédures

Exemples de paramètres IN:

```
CREATE OR REPLACE PROCEDURE modifier_salaire
  (Emp_id IN employees.employee_id%TYPE)
IS
BEGIN
  UPDATE employees
  SET    salaire = salaire * 1.10
  WHERE  employee_id = Emp_id;
END modifier_salaire;
```

Les procédures

Exemples de paramètres OUT



Les procédures

Exemples de paramètres OUT

query_emp.sql

```
CREATE OR REPLACE PROCEDURE query_emp
  (p_id      IN    employees.employee_id%TYPE,
   p_nom      OUT   employees.nom%TYPE,
   p_salaire  OUT   employees.salaire%TYPE,
   p_comm     OUT   employees.commission%TYPE)
IS
BEGIN
  SELECT    nom, salaire, commission
  INTO      p_nom, p_salaire, p_comm
  FROM      employees
  WHERE     employee_id = p_id;
END query_emp;
```

Les procédures

Visualiser des paramètres OUT

- Déclarer les variables, exécuter la procédure QUERY_EMP, puis imprimer la valeur de la variable globale G_NAME

```
VARIABLE g_name      VARCHAR2 (25) ;  
VARIABLE g_sal        NUMBER;  
VARIABLE g_comm        NUMBER;  
  
EXECUTE query_emp(171, :g_name, :g_sal, :g_comm);  
  
PRINT g_name;
```

Les procédures

Exemples de paramètres IN OUT

Environnement appelant

'2126330575'

'(212)633-0575' p_phone_no

```
CREATE OR REPLACE PROCEDURE format_phone
  (p_phone_no IN OUT VARCHAR2)
IS
BEGIN
  p_phone_no := '(' || SUBSTR(p_phone_no,1,3) ||
                ')' || SUBSTR(p_phone_no,4,3) ||
                '-' || SUBSTR(p_phone_no,7,4);
END format_phone;
```

Les procédures

Visualiser des paramètres IN OUT

```
VARIABLE phone VARCHAR2(15)

EXECUTE      :phone := '212668448477';

EXECUTE format_phone (:phone)
PRINT phone
```

Résultat :

PHONE

(212) 668-448477

Les procédures

Déclarer des sous-programmes

```
CREATE OR REPLACE PROCEDURE leave_emp2
  (p_id IN employees.employee_id%TYPE)
IS
  PROCEDURE log_exec
  IS
  BEGIN
    INSERT INTO log_table (user_id, log_date)
    VALUES (USER, SYSDATE);
  END log_exec;
BEGIN
  DELETE FROM employees
  WHERE employee_id = p_id;
  log_exec;
END leave_emp2;
/
```

Les procédures

Appeler une procédure depuis un bloc PL/SQL anonyme

```
DECLARE
    v_id NUMBER := 163;
BEGIN
    Modifier_salaire(v_id);    --Appel procedure
    COMMIT;
    ...
END;
```

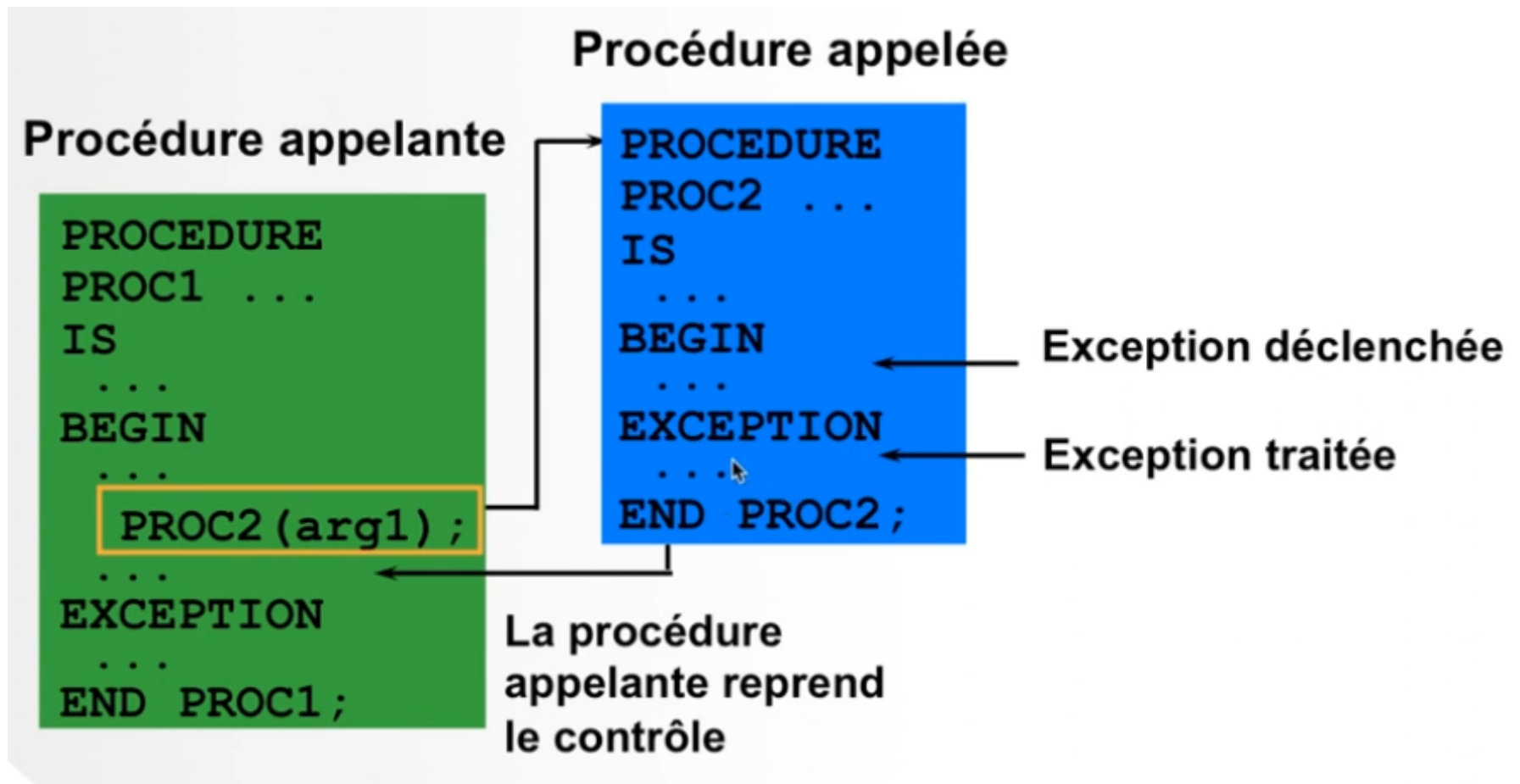
Les procédures

Appeler une procédure depuis une autre procédure

```
CREATE OR REPLACE PROCEDURE process_emps
IS
    CURSOR emp_cursor IS
        SELECT employee_id FROM employees;
BEGIN
    FOR emp_rec IN emp_cursor
    LOOP
        Modifier_salaire(emp_rec.employee_id);
    END LOOP;
    COMMIT;
END process_emps;
/
```

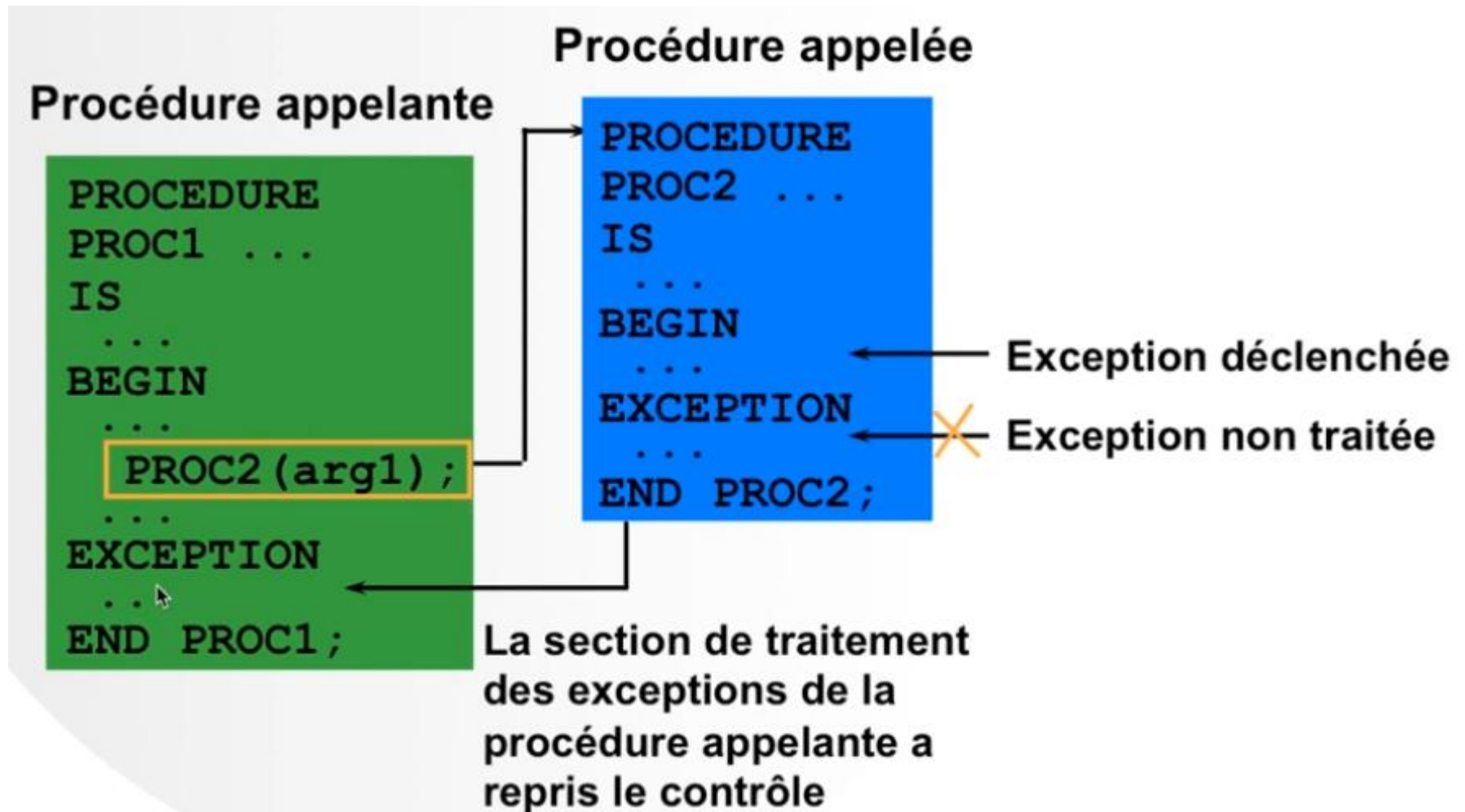
Les procédures

Exceptions traitées



Les procédures

Exceptions non traitées



Les procédures

Supprimer une procédure dans la base de données

Syntaxe:

```
DROP PROCEDURE procedure_name
```

Exemple:

```
DROP PROCEDURE Modifier_salaire;
```

Synthèse

- Une procédure est un sous-programme qui exécute une action
- Vous pouvez créer des procédures en utilisant la commande CREATE PROCEDURE
- Vous pouvez compiler et enregistrer une procédure dans la base de données
- Il existe trois modes de paramètre: IN, OUT, IN OUT
- Les procédures peuvent être appelées à partir de n'importe quel outil ou langage prenant en charge le langage PL/SQL
- Vous devez être conscient de l'impact des exceptions traitées et non traitées sur les transactions et les procédures appelantes
- Vous pouvez supprimer des procédures de la base de données en utilisant la commande DROP PROCEDURE

Les procédures

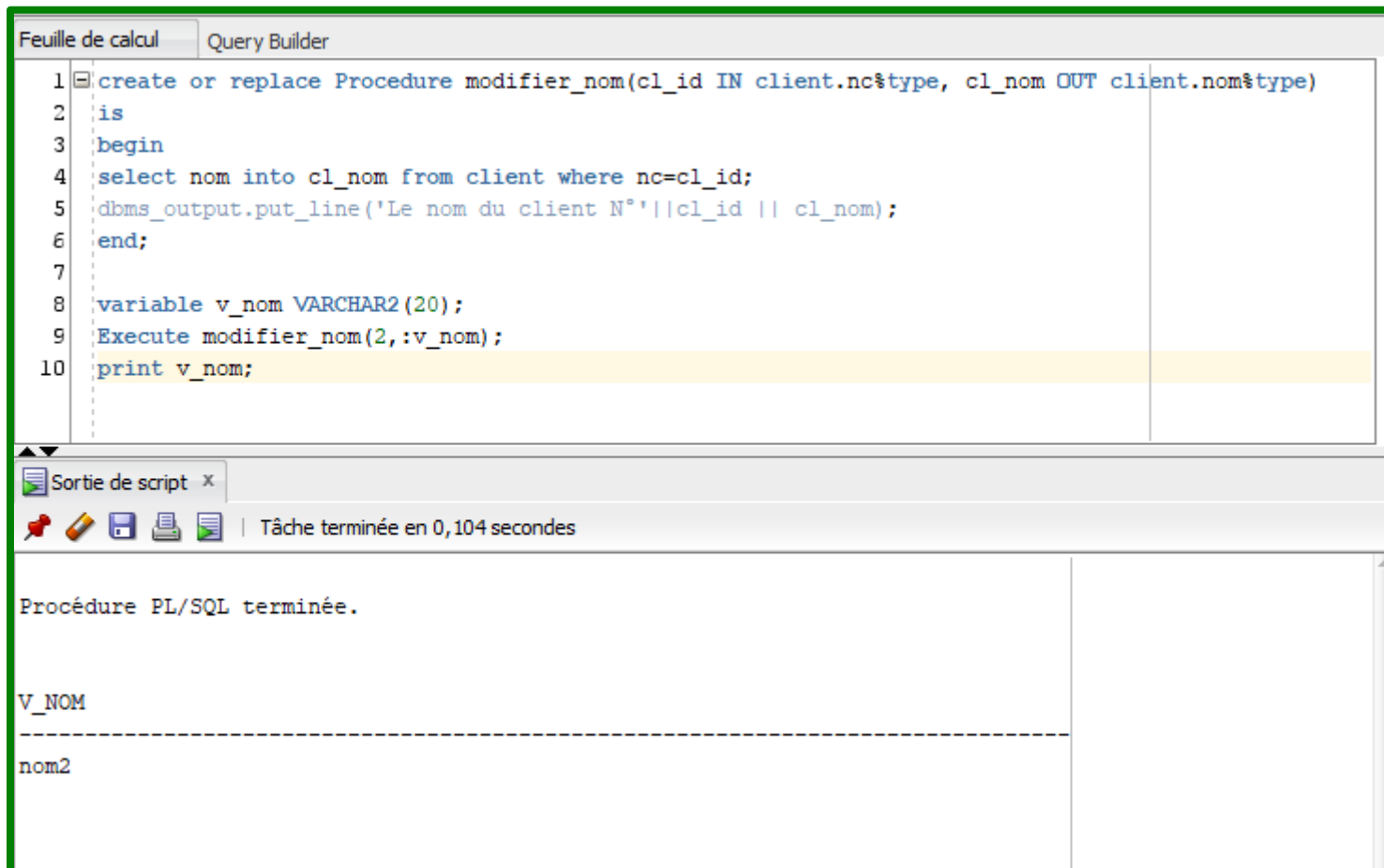
Exemple de paramètres IN

	Feuille de calcul	Query Builder
1		<code>create or REPLACE Procedure modifier_nom(cl_id client.nc\$type)</code>
2		<code>is</code>
3		<code>begin</code>
4		<code>update client set nom='rachid' where nc=cl_id;</code>
5		
6		<code>end;</code>
7		

	Feuille de calcul	Query Builder
1		<code>declare</code>
2		<code>entier integer:=2;</code>
3		<code>begin</code>
4		
5		<code>modifier_nom(entier);</code>
6		<code>end;</code>

Les procédures

Exemple de paramètres IN et OUT



The screenshot displays the Oracle SQL Developer interface. The top pane, titled 'Query Builder', contains the following PL/SQL code:

```
1 create or replace Procedure modifier_nom(cl_id IN client.nc$type, cl_nom OUT client.nom$type)
2 is
3 begin
4 select nom into cl_nom from client where nc=cl_id;
5 dbms_output.put_line('Le nom du client N°'||cl_id || cl_nom);
6 end;
7
8 variable v_nom VARCHAR2(20);
9 Execute modifier_nom(2,:v_nom);
10 print v_nom;
```

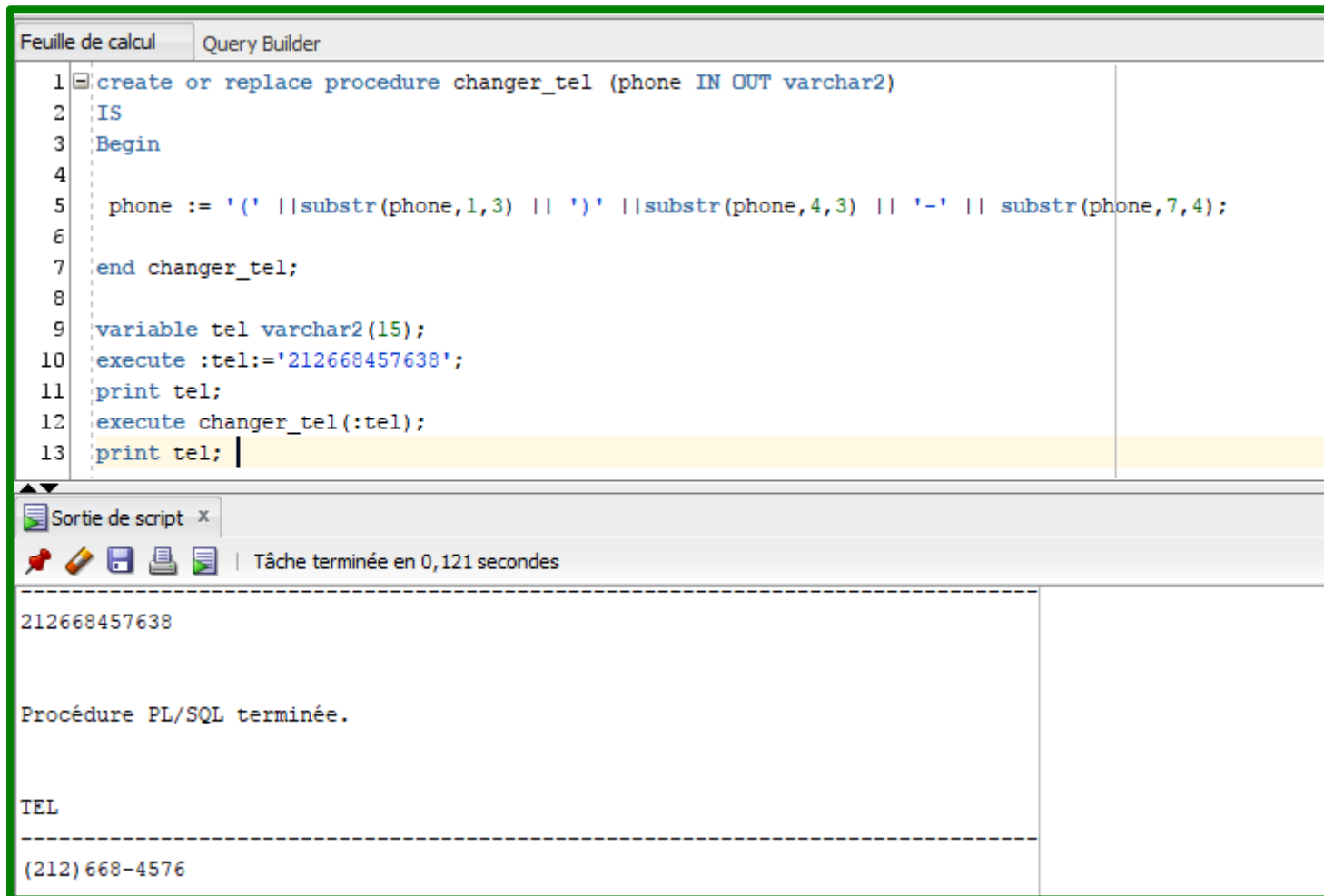
The bottom pane, titled 'Sortie de script', shows the execution results:

```
Procédure PL/SQL terminée.

V_NOM
-----
nom2
```

Les procédures

Exemple de paramètres IN OUT



The screenshot shows a SQL Query Builder window with a script editor and a results pane. The script defines a procedure `changer_tel` that takes a phone number as an IN OUT parameter and reformats it. The procedure is executed with the phone number '212668457638'. The results pane shows the output of the procedure, which is '(212) 668-4576'.

```
1 create or replace procedure changer_tel (phone IN OUT varchar2)
2 IS
3 Begin
4
5     phone := '(' || substr(phone,1,3) || ')' || substr(phone,4,3) || '-' || substr(phone,7,4);
6
7 end changer_tel;
8
9 variable tel varchar2(15);
10 execute :tel:='212668457638';
11 print tel;
12 execute changer_tel(:tel);
13 print tel;
```

Sortie de script x

Tâche terminée en 0,121 secondes

212668457638

Procédure PL/SQL terminée.

TEL

(212) 668-4576

Les procédures

Exemple de paramètres IN OUT

Appeler une procédure depuis un bloc PL/SQL anonyme

```
create or replace procedure changer_tel (phone IN OUT varchar2)
IS
Begin

    phone := '(' || substr(phone,1,3) || ')' || substr(phone,4,3) || '-' || substr(phone,7,4);

end changer_tel;
```

```
set SERVEROUTPUT ON
declare

tel varchar2(15):='212668234356';

begin
dbms_output.put_line('Numéro tel avant :'||tel);
changer_tel(tel);
dbms_output.put_line('Numéro tel avant :'||tel);
end;
```



PL/SQL

Les fonctions

Objectifs



- Décrire les différentes utilisations des fonctions
- Créer des fonctions stockées
- Appeler une fonction
- Supprimer une fonction
- Faire la distinction entre une procédure et une fonction

Présentation des fonctions stockées



- Une fonction est un bloc PL/SQL nommé qui **renvoie une valeur**
- Une fonction peut être stockée en tant qu'objet de schéma dans la base de données en vue **d'exécutions répétées**
- Appeler une fonction
- Une fonction est **appelée dans une expression**
- **Faire la distinction entre une procédure et une fonction**

Les fonctions

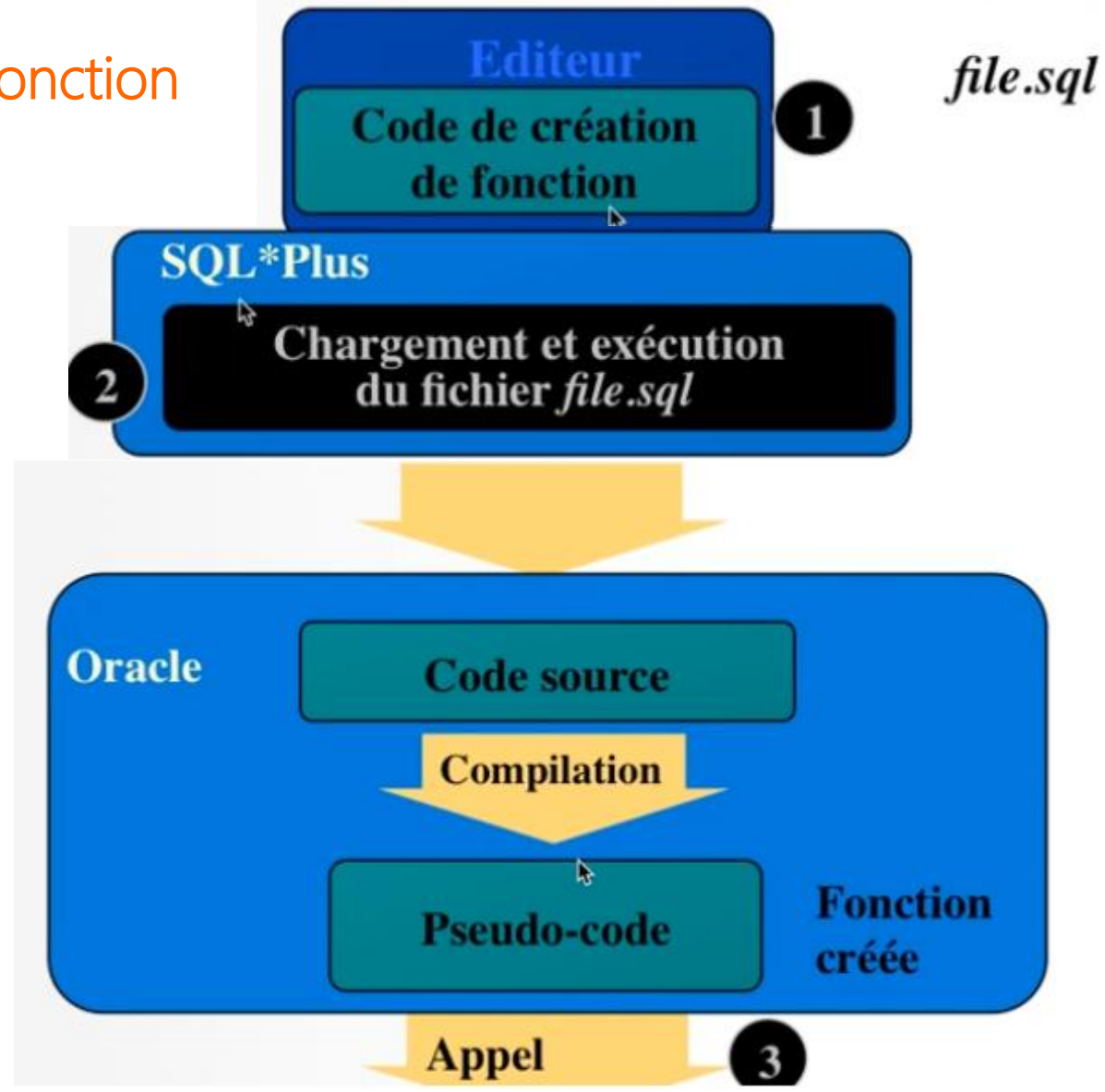
-
- Syntaxe pour la création de fonctions

```
CREATE [OR REPLACE] FUNCTION function_name
  [(parameter1  datatype1,
   parameter2   datatype2,
   . . .)]
RETURN datatype -- ne doit pas inclure de spécification de taille
IS|AS
PL/SQL Block;
```

Le bloc PL/SQL doit comporter au moins une instruction RETURN.

Les fonctions

- Création de la fonction



Les fonctions

Exemple de création d'une fonction stockée

```
CREATE OR REPLACE FUNCTION get_salaire
    (p_id IN employees.employee_id%TYPE)
    RETURN NUMBER
IS
    v_salaire employees.salaire%TYPE :=0;
BEGIN
    SELECT salaire
    INTO    v_salaire
    FROM    employees
    WHERE   employee_id = p_id;
    RETURN v_salaire;
END get_salaire;
```

Les fonctions

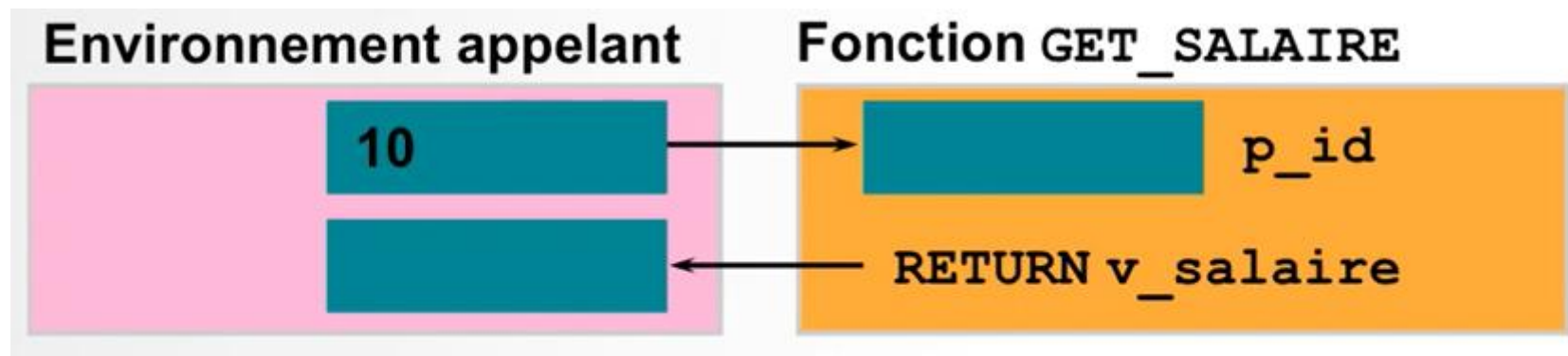
- Exécuter des fonctions

- Appeler une fonction dans une expression PL/SQL
- Créer une variable destinée à recevoir la valeur renvoyée
- Exécuter la fonction
- La valeur renvoyée par l'instruction RETURN sera placée dans la variable

RQ: Eviter d'utiliser les modes OUT et IN OUT avec les fonctions

Les fonctions

Exemple d'exécution de fonctions



- 1 → Charger et exécuter le fichier get_salary.sql pour créer la fonction
- 2 →

```
VARIABLE salaire NUMBER
```
- 3 →

```
EXECUTE :salaire := get_salaire(10)
```
- 4 →

```
PRINT salaire
```

Les fonctions

Avantages des fonctions définies par l'utilisateur dans les expressions SQL

- Elles **complètent le langage SQL** en permettant de réaliser des traitements qui seraient trop complexes, voire impossibles en SQL.
- **Utilisées dans la clause WHERE** pour filtrer les données, elles peuvent s'avérer plus efficaces qu'un filtrage au sein de l'application
- Elles permettent de manipuler les chaînes de caractères

Les fonctions

Exemple d'appel de fonctions dans des expressions SQL

```
CREATE OR REPLACE FUNCTION tax(p_value IN NUMBER)
RETURN NUMBER IS

BEGIN
    RETURN (p_value * 0.08);
END tax;
```

Appel fonction

```
SELECT employee_id, nom, salaire, tax(salaire)
FROM employees WHERE department_id = 100;
```

Les fonctions



Emplacements d'appel des fonctions définies par l'utilisateur

- Liste de sélection d'une commande **SELECT**
- Condition des clauses **WHERE** et **HAVING**
- Condition **ORDER BY** et **GROUP BY**
- Clause **VALUES** de la commande **INSERT**
- Clause **SET** de la commande **UPDATE**

Les fonctions

Exemple appel fonction

```
SELECT employee_id, tax(salaire)
FROM employees
WHERE tax(salaire) > (SELECT tax(salaire)
                      FROM employees
                      WHERE department_id=30
                      )
ORDER BY tax(salaire) DESC;
```

Restrictions relatives à l'appel de fonctions à partir d'expressions SQL

Pour pouvoir être appelée depuis des expressions SQL, une fonction définie par l'utilisateur doit :

- Être une **fonction stockée** (ce n'est pas le cas pour les procédures stockées.)
- Accepter uniquement des **paramètres IN**
- En tant que paramètres, accepter uniquement des **types de données SQL valides** (et non des types spécifiques au langage PL/SQL)
- **Renvoyer des types de données SQL** valides et non des types spécifiques au langage PL/SQL
- Les fonctions appelées depuis des expressions SQL (SELECT, UPDATE, DELETE en parallèle) **ne peut modifier** aucune table de la BDD

Les fonctions

- Supprimer une fonction stockée

```
DROP FUNCTION function_name
```

Exemple :

```
DROP FUNCTION get_salaire;
```

Function dropped.

- Tous les **privilèges** accordés à une fonction sont **annulés** lorsque celle-ci est **supprimée**.
- Lorsque la syntaxe **CREATE OR REPLACE** est utilisée, la fonction est supprimée et recréée, mais dans ce cas, les **privilèges** accordés sur la fonction **ne sont pas affectés**.

Synthèse

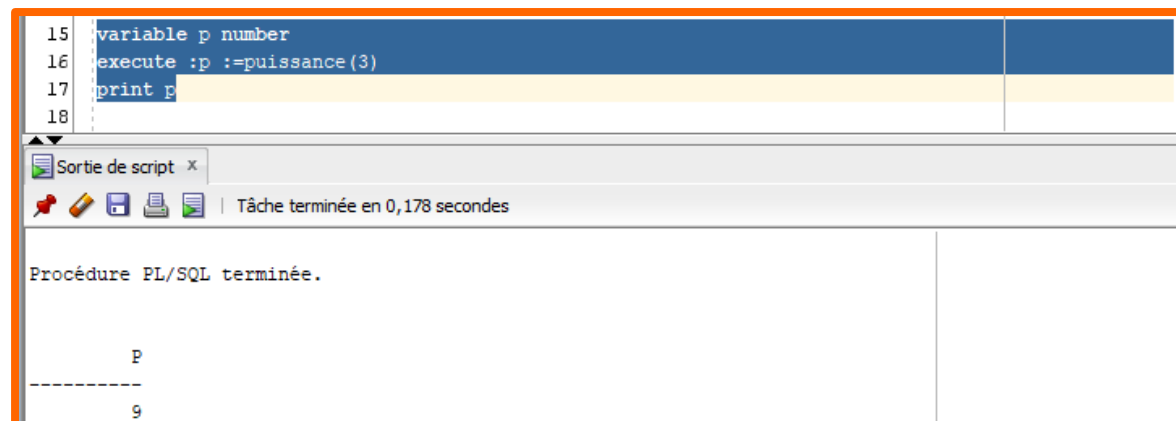
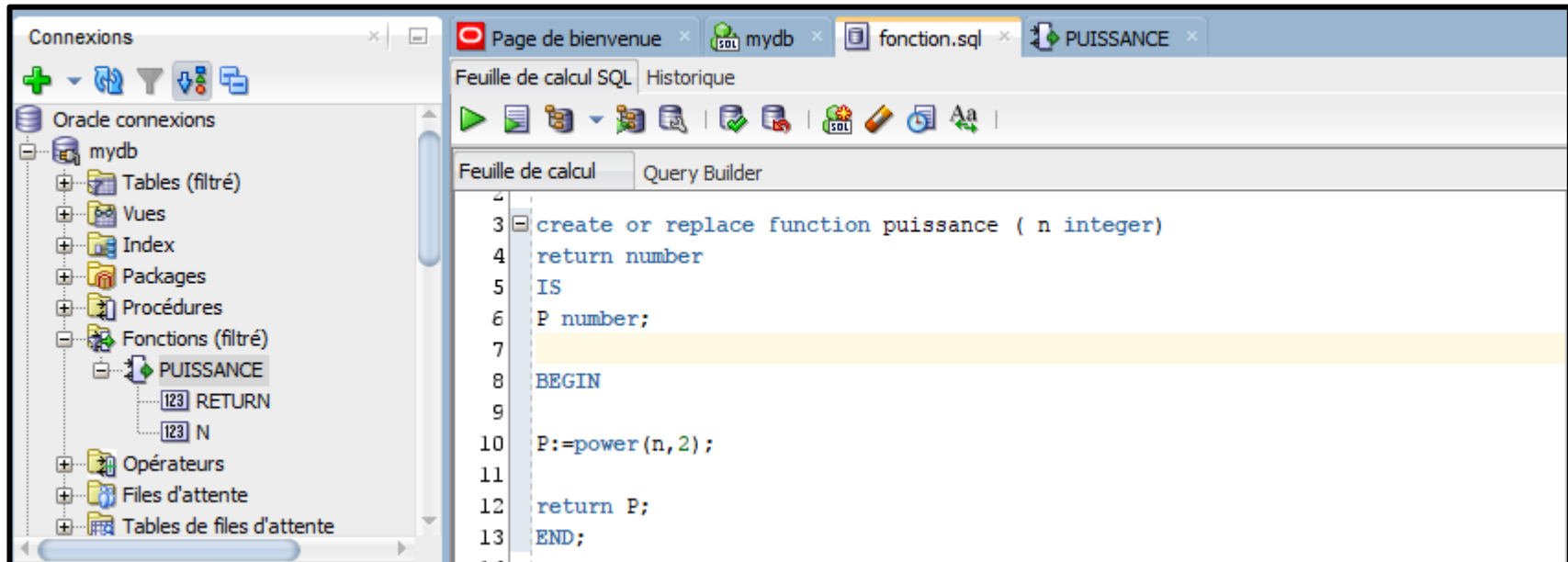
- Une fonction est **un bloc PL/SQL** nommé qui doit renvoyer une valeur
- La syntaxe **CREATE FUNCTION** permet de créer une fonction
- Une fonction stockée dans la base de données **peut être appelée dans les instructions SQL**
- La syntaxe **DROP FUNCTION** permet de **supprimer une fonction de la base de données**
- En règle générale, une procédure permet **d'exécuter une action** tandis qu'une fonction permet de **calculer une valeur**
- Vous pouvez **supprimer des procédures de la base de données** en utilisant la commande **DROP PROCEDURE**

Comparer les procédures et les fonctions

Procédures	Fonctions
S'exécutent en tant qu'instruction PL/SQL	Sont appelées dans une expression
Ne contiennent pas de clause RETURN dans l'en-tête	Doivent contenir une clause RETURN dans l'en-tête
Peuvent transférer zéro, une ou plusieurs valeurs	Doivent renvoyer une seule valeur
Peuvent contenir une instruction RETURN	Doivent contenir au moins une instruction RETURN

Les fonctions

Exemple de fonction puissance



Les fonctions

Exemple de fonction / Procedure / bloc Anonyme

```
1  /* une fonction qui calcule la puissance en 2 d'une valeur */
2
3  create or replace function puissance ( n integer)
4  return number
5  IS
6  P number;
7
8  BEGIN
9
10 P:=power(n,2);
11
12 return P;
13 END;
```

```
1  create or replace procedure  afficheinfoClient( n integer)
2  IS
3  var_nom client.nom%type;
4  var_prenom client.prenom%type;
5
6  BEGIN
7  if (puissance(n)<4) then
8  select nom,prenom into var_nom,var_prenom from client where nc=n;
9  dbms_output.put_line('le nom du client : '||var_nom);
10 dbms_output.put_line('le prénom du client : '||var_prenom);
11 end if;
12 END;
```

```
1  create or replace procedure  afficheinfoClient( n integer)
2  IS
3  var_nom client.nom%type;
4  var_prenom client.prenom%type;
5
6  BEGIN
7  if (puissance(n)<4) then
8  select nom,prenom into var_nom,var_prenom from client where nc=n;
9  dbms_output.put_line('le nom du client : '||var_nom);
10 dbms_output.put_line('le prénom du client : '||var_prenom);
11 end if;
12 END;
```



PL/SQL

Les packages

Objectifs

- Décrire des **packages** et répertorier leurs éventuels composants
- Créer un package regroupant des variables, constantes, exceptions, procédures, fonctions et curseurs associées
- Désigner une structure de package comme publique ou privée
- Appeler une structure de package
- Décrire l'utilisation d'un package sans corps

Présentation des packages

Les packages

- Regroupent des types **PL/SQL**, des éléments et des sous-programmes présentant une relation logique
- Sont constitués de deux éléments :
 - Spécification (**déclaration**)
 - Corps (**déclaration**)
- Ne peuvent pas être appelés, paramétrés ou imbriqués
- Permettent au serveur Oracle de lire simultanément plusieurs objets en mémoire

Composants d'un package

Spécification
du package

Déclaration de la
procédure A

Variable publique

Procédure publique

Corps du
package

Définition de la
procédure B

Variable privée

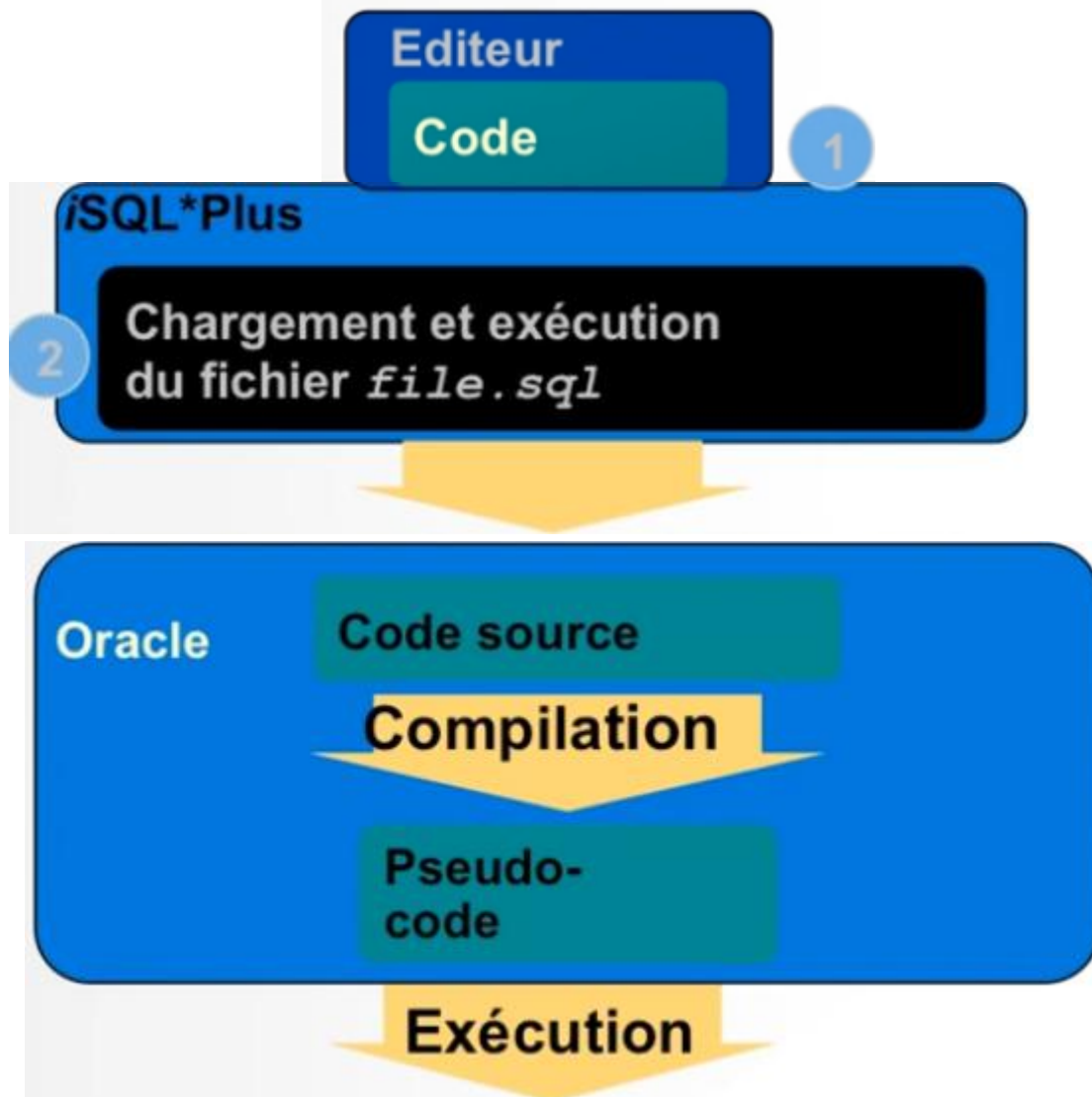
Procédure privée

Définition de la
procédure A

Procédure publique

Variable locale

Développer un package



Développer un package



- L'enregistrement du texte de l'instruction **CREATE PACKAGE** dans deux fichiers SQL distincts facilite les modifications ultérieures du package.
 - Spécification : **CREATE PACKAGE**
 - Corps: **CREATE PACKAGE BODY**

Créer la spécification du package

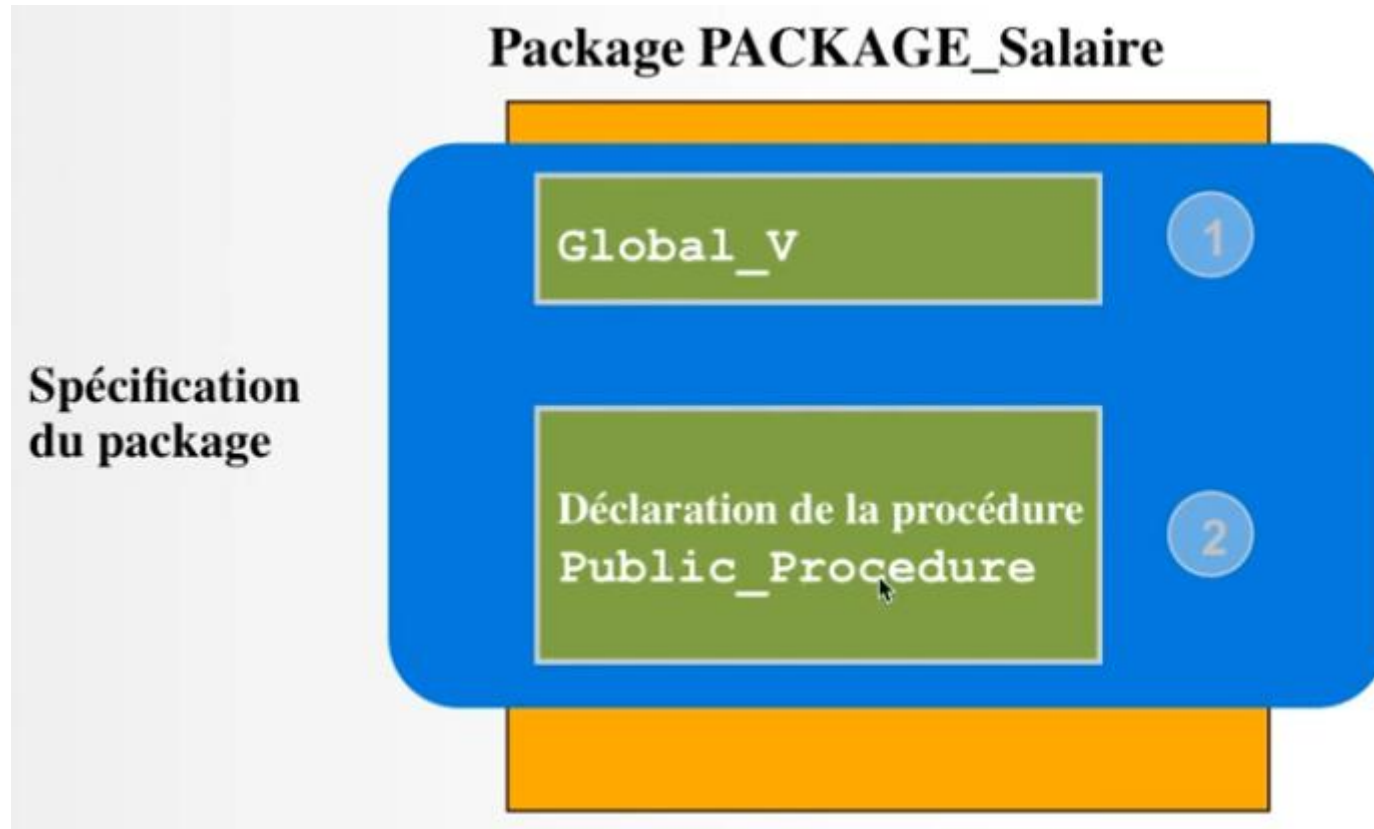
Syntaxe:

```
CREATE [OR REPLACE] PACKAGE package_name
IS | AS
    définitions des types utilisés dans le package;
    Prototypes de toutes les procédures et fonctions du package;

END package_name;
```

- L'option **REPLACE** supprime et recrée la spécification du package
- Toutes les structures déclarées dans une spécification de package peuvent être visibles par les utilisateurs disposant de privilèges sur le package

Les structures publiques



Les structures publiques

Exemple : spécifications package

```
CREATE OR REPLACE PACKAGE PACKAGE_Salaire
IS
    global_v NUMBER := 0.10;  --initialisé à 0.10
    PROCEDURE Public_procedure(p_comm IN NUMBER);
END PACKAGE_Salaire;
/
```

Package created.

- **g_var** est une variable globale dont la valeur d'initialisation est 0,10.
- **Public_procedure** est une procédure publique implémentée dans le corps du package

Les structures publiques

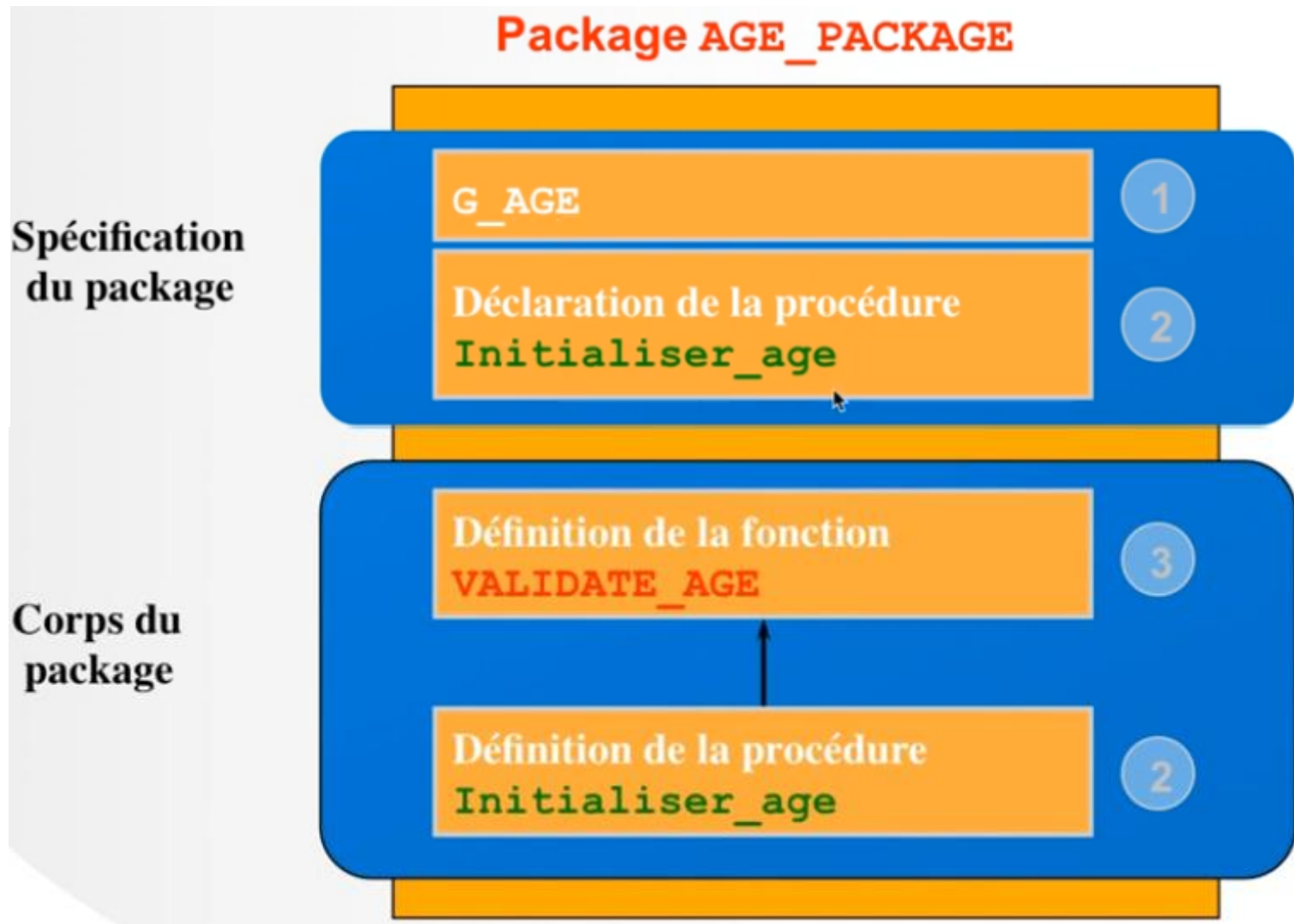
Syntaxe:

```
CREATE [OR REPLACE] PACKAGE BODY package_name
IS | AS
    Déclaration de variables privées;

END package_name;
```

- L'option **REPLACE** supprime et recrée le corps du package
- Les identificateurs définis exclusivement dans le corps du package sont des structures privées. Ils ne sont pas visibles à l'extérieur du corps du package
- Toutes les structures privées doivent être déclarées avant d'être utilisées dans les structures publiques

Structures publiques et privées



Création de corps de package

Exemple : age_pack.sql

```
CREATE OR REPLACE PACKAGE BODY AGE_PACKAGE
IS
    FUNCTION validate_age (Par_age IN NUMBER)
        RETURN BOOLEAN
    IS
        v_max_age      NUMBER;
    BEGIN
        SELECT      MAX(age)
        INTO        v_max_age
        FROM        client;
        IF      Par_age > v_max_age THEN RETURN(FALSE);
        ELSE      RETURN(TRUE);
        END IF;
    END validate_age;
    ...
```

Création de corps de package

Exemple : age_pack.sql

```
PROCEDURE  initialiser_age (p_age    IN  NUMBER)
IS
BEGIN
  IF  validate_age (p_age) <
  THEN    g_age:=p_age;
        DBMS_OUTPUT.PUT_LINE('Age n''est pas valide' );
  END IF;
END  initialiser_age ;
END age_package;
/
```

Appel d'une fonction depuis une procédure du même package.

Structures publiques et privées

Spécification du package

```
Create or replace package test
Is
Var1 Constant number:=10;
Var2 varchar2(2):= 'ENSA';
Procedure affichage;
End;
```

Corps du package

```
Create or replace package body test
Is
Var3 varchar2(2):= 'FST';

Procedure getVar
Is
Begin
DBMS_OUTPUT.PUT_LINE(Var1) ;
DBMS_OUTPUT.PUT_LINE(Var3) ;
End;

Procedure affichage
Is
Var4 varchar2(2):= 'EST';
Begin
getVar();
DBMS_OUTPUT.PUT_LINE(Var2) ;
DBMS_OUTPUT.PUT_LINE(Var3) ;
DBMS_OUTPUT.PUT_LINE(Var4) ;
End;
End;
```

Appeler des structures de package

Appeler une procédure de package depuis SQL*Plus

```
EXECUTE nom_package.nom_procedure(valeurs  
paramètres..)
```

Appeler une procédure de package dans un autre schéma

```
EXECUTE scott.nom_package.nom_procedure (valeurs  
paramètres..)
```

Déclarer un package sans corps

```
CREATE OR REPLACE PACKAGE global_var IS
  PI CONSTANT  NUMBER  :=  3.14;
  AGE CONSTANT  NUMBER  :=  30;
  R CONSTANT  NUMBER  :=  10;
END global_var ;
/

EXECUTE DBMS_OUTPUT.PUT_LINE('age  = '|| global_var.age)
EXECUTE DBMS_OUTPUT.PUT_LINE('Air d'un disque  = '||
2*global_var.PI*global_var.R)
```

Référencer une variable publique depuis une procédure autonome

Exemple:

```
CREATE OR REPLACE PROCEDURE Air_Disque  
(air OUT NUMBER)  
IS  
BEGIN  
    air:= 2*global_var.PI*global_var.R;  
END;  
/
```

```
VARIABLE air NUMBER  
EXECUTE Air_Disque ( :air)  
PRINT air
```

Supprimer des packages

Utilisez la syntaxe suivante pour supprimer la spécification et le corps du package

```
DROP PACKAGE package_name;
```

Utilisez la syntaxe suivante pour supprimer le corps du package

```
DROP PACKAGE BODY package_name;
```

Supprimer des packages

Pour afficher les informations sur un package

```
Select * from user_objects where object_name='nom package'
```

Pour afficher le code source d'une spécification

```
Select * from user_source  
Where name='nom package' and type = 'PACKAGE'
```

Pour afficher le code source d'un Corps (Body)

```
Select * from user_source  
Where name='nom package' and type = 'PACKAGE BODY'
```

Avantages liés aux packages

- **Modularité** : encapsule les structures associées
- **Conception simplifiée des applications** : la spécification et le corps sont codés et compilés séparément
- **Masquage d'informations** :
 - Seules les déclarations contenues dans la spécification du package sont **visibles et accessibles aux applications**
 - Les structures privées du corps du package sont **masquées et inaccessibles**
 - L'ensemble du code est **masqué dans le corps du package**

Avantages liés aux packages



- Performances accrues :
 - L'ensemble du package est chargé en mémoire la première fois que celui-ci est référencé
 - Une seule copie est chargée en mémoire pour l'ensemble des utilisateurs
- Surcharge : plusieurs sous-programmes portant le même nom

Synthèse



- Optimiser l'organisation, la gestion, la sécurité et les performances en utilisant des packages
- Regrouper les procédures et les fonctions associées au sein d'un package
- Modifier un corps de package sans affecter sa spécification
- Définir un accès sécurisé à l'ensemble du package
- Masquer le code source aux utilisateurs
- Charger l'ensemble du package en mémoire au premier appel
- Réduire les accès au disque pour les appels ultérieurs

Exercice

Soit le schéma relationnel suivant :

❖ **Client** (*Num_cl*, *Nom*, *prénom*, *age*, *ville*)

- Ecrire un package contenant la spécification suivante:
 - Une procédure « **Add_Client** » pour insérer les informations d'un client
 - Une procédure « **Add_client** » pour insérer les informations d'un client sauf la ville
 - Une procédure « **get_client** » qui affiche les informations d'un client donné
 - Une fonction « **get_age** » qui retourne l'âge d'un client donné
 - Une procédure « **delete_client** » qui supprime un client donné
 - Une procédure « **delete_client** » pour supprimer les clients dont l'âge est dans la liste {20,30,40}
- Implémenter le corps de toutes les spécifications:

Exercice

The screenshot displays a SQL IDE interface with the following components:

- Top Bar:** Contains tabs for 'Page de bienvenue', 'Package_Client.sql', and 'mydb'. Below the tabs is a toolbar with icons for running, saving, and other actions, along with a timer showing '0,28600001 secondes'.
- Feuille de calcul SQL / Historique:** The main editor area showing the SQL code for creating and replacing the package 'test_client'. The code includes several procedures and a function.
- Feuille de calcul / Query Builder:** A section for building queries, currently empty.
- Sortie de script:** A section at the bottom showing the output of the script execution, indicating that the package 'TEST_CLIENT' was successfully compiled.

```
2 create or replace package test_client
3 is
4 procedure add_client(id client.nc%type, fname client.nom%type, lname client.prenom%type,
5                      city client.ville%type, age client.age%type);
6 procedure add_client(id client.nc%type, fname client.nom%type, lname client.prenom%type,
7                      age client.age%type);
8 procedure getclient(id client.nc%type);
9 function getage(id client.nc%type) return number;
10 procedure deleteclient(id client.nc%type);
11 procedure deleteclient;
12 end test_client;
13
14 /* Corps du package test_client */
```

Sortie de script x

Tâche terminée en 0,286 secondes

Elément Package TEST_CLIENT compilé

Exercice

Feuille de calcul SQL Historique

mydb

Feuille de calcul Query Builder

```
12 end test_client;
13
14      /* Corps du package test_client */
15 create or replace package body test_client
16 is
17 procedure add_client(id client.nc%type, fname client.nom%type, lname client.prenom%type,
18                      city client.ville%type, age client.age%type)
19 is
20 begin
21     insert into client(nc, nom, prenom, ville, age) values (id, fname, lname, city, age);
22 end;
23
24 procedure add_client(id client.nc%type, fname client.nom%type, lname client.prenom%type,
25                      age client.age%type)
26 is
27 begin
28     insert into client(nc, nom, prenom, age) values (id, fname, lname, age);
```

Sortie de script x

Tâche terminée en 0,112 secondes

Élément Package Body TEST_CLIENT compilé

Exercice

The screenshot displays the Oracle SQL Developer environment. The main window is titled "Feuille de calcul SQL" and "Historique". The toolbar includes icons for running, saving, and other database operations, along with a timer showing "0,60699999 secondes". The database connection is set to "mydb".

The "Feuille de calcul" tab is active, showing a PL/SQL script in the "Query Builder" view. The script is as follows:

```
72 end loop;  
73 end;  
74 end;  
75 /* Tester le package */  
76  
77 execute test_client.add_client(7,'nom7','prenom7', 'ville7', 70);  
78  
79  
80  
81  
82  
83  
84
```

The line 77, `execute test_client.add_client(7,'nom7','prenom7', 'ville7', 70);`, is highlighted in yellow.

Below the script editor, a "Sortie de script" window is open, showing the execution result:

Procédure PL/SQL terminée.

Exercice

The screenshot displays the Oracle SQL Developer environment. The top toolbar includes icons for running, saving, and other database operations, along with a timer showing 0,11 secondes. The database connection is set to 'mydb'. The main window is titled 'Feuille de calcul SQL' and 'Historique'. Below this, the 'Feuille de calcul' tab is active, showing a PL/SQL script. The script contains several lines of code, with the last two lines highlighted in yellow:

```
72 end loop;  
73 end;  
74 end;  
75 /* Tester le package */  
76  
77 execute test_client.add_client(7,'nom7','prenom7', 'ville7', 70);  
78  
79 execute test_client.add_client(8,'nom8','prenom8', 30);  
80  
81  
82  
83  
84
```

The bottom window, titled 'Sortie de script', shows the execution results. It contains two lines of text, both indicating that the PL/SQL procedure was completed successfully:

```
Procédure PL/SQL terminée.  
  
Procédure PL/SQL terminée.
```